

Computer Graphics and Visualization

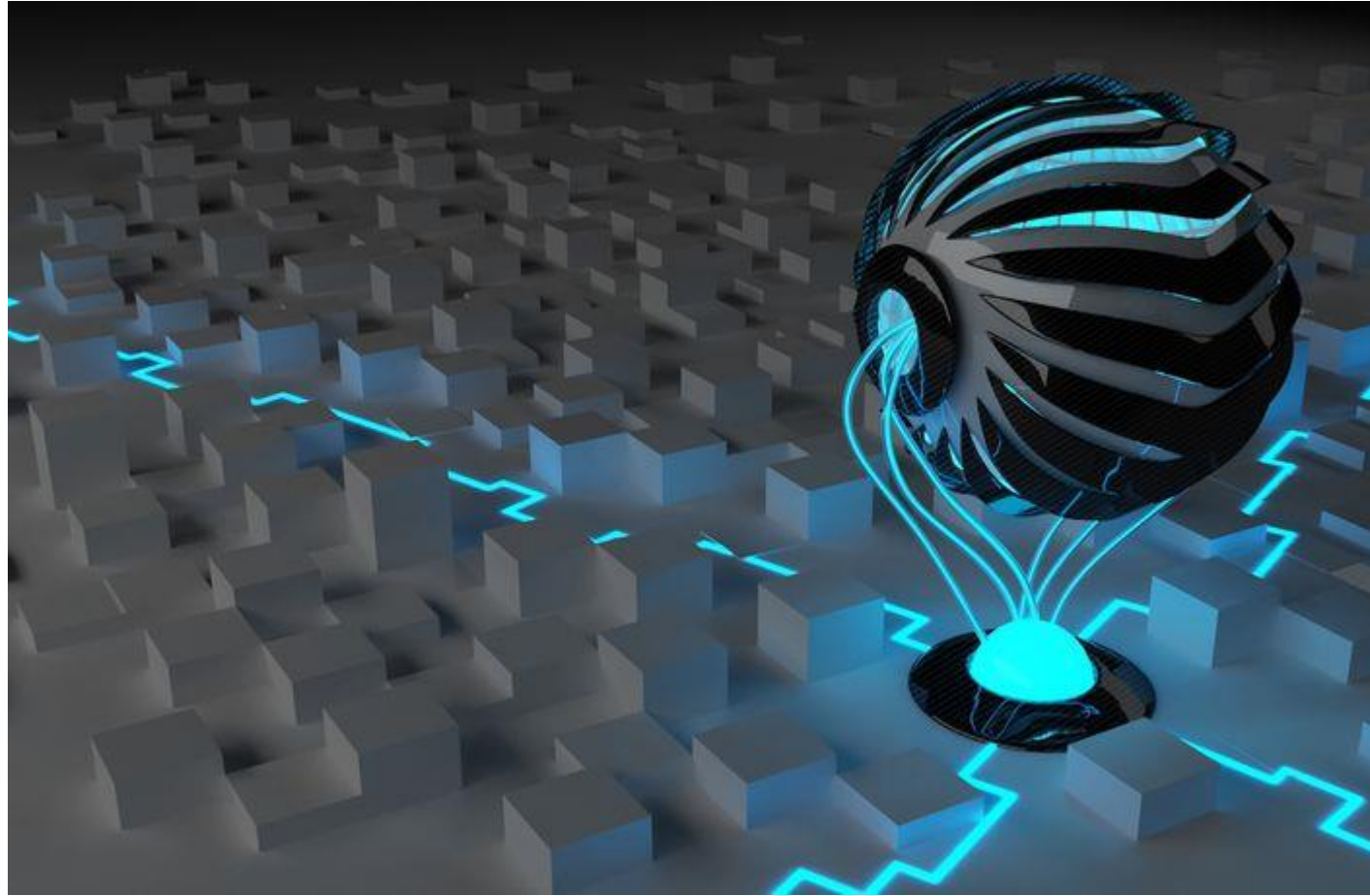
Chapter 4

2DCurve Modeling and Surface Modeling

Prepared by:

Asst. Prof. Sanjivan Satyal

Asst. Prof. Sanjivan Satyal



Content: Curve Modeling and Surface Modelling [4 hours]

- 4.1. Introduction to Parametric cubic Curves, Splines, Bezier curves
- 4.2. Surface modeling
 - 4.2.1. Polygon surface, vertex table, edge table, polygon table, surface normal and spatial orientation of surfaces

Curves

- ❑ Many objects we want to model are not straight e.g. Text, sketches, etc.
- ❑ A curve is defined as a continuous line with bent but not with sharp turns
 - ❑ Arc, circle, ellipse, sphere, cylinder, parabola
- ❑ A curve is an infinitely large set of points. Each point has two neighbours except endpoints



- ❑ Commonly encountered curves include conics, trigonometric and exponential functions, probability distributions, general polynomials, and spline functions
- ❑ Curves are useful as
 - ❑ Exact boundary representation for some objects
 - ❑ More concise representation than polygonal mesh
 - ❑ Easier to model with and specify for many man-made objects and machine parts
 - ❑ object modelling, animation path specifications, data and function graphing, and other graphics applications

Types of Curves

- ❑ A curve is an infinitely large set of points. Each point has two neighbours except endpoints
- ❑ Curves can be broadly classified into three categories:
 - ❑ Explicit
 - ❑ Implicit
 - ❑ Parametric curves

1. Explicit Curves

- ❑ The dependent variable is given explicitly in terms of the independent variable as:
 - ❑ $y = f(x)$
 - ❑ $y = mx + c$
- ❑ For each single value of x , only single value of y is computed

2. Implicit Curves

- ❑ Implicit curve representations define the set of points on a curve by employing a procedure that can test to see if a point is on the curve
- ❑ Usually, an implicit curve is defined by an implicit function of the form $f(x, y) = 0$
- ❑ Eg. A common example is the circle, whose implicit representation is $x^2 + y^2 - r^2 = 0$
- ❑ For each single value of x , multiple values of y are computed

3. Parametric curve

- ❑ The explicit and implicit curve representations can be used only when the function is known
- ❑ Curves having parametric form are called parametric curves
- ❑ In practice the parametric curves are used

❑ Here, x and y are defined in terms of third variable in parametric form

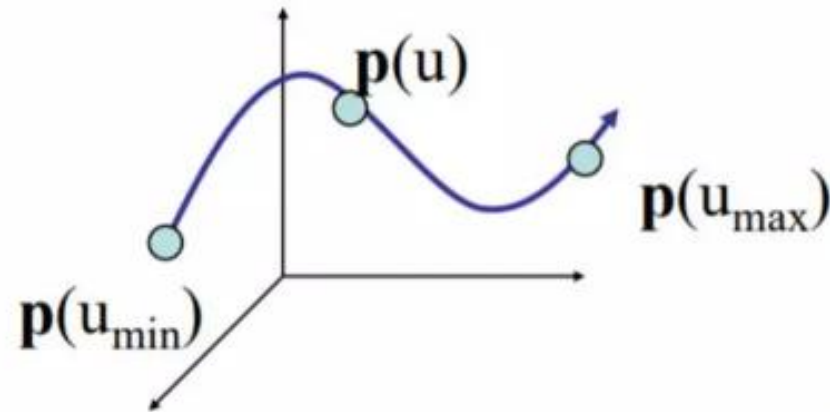
$$x = f_x(u)$$

$$y = f_y(u) \text{ where } u \text{ is parameter}$$

❑ A two-dimensional parametric curve has the following form:

$$\text{❑ } P(t) = f(t), g(t) \quad \text{or} \quad P(t) = x(t), y(t)$$

- The functions f and g become the (x, y) coordinates of any point on the curve, and the points are obtained when the parameter t (or u) is varied over a certain interval $[a, b]$, normally $[0, 1]$



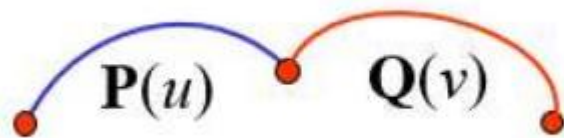
Curve Continuity

- To ensure a smooth transition from one section of a piecewise parametric spline to the next we can impose various **continuity conditions** at the connection points:

- ☐ Parametric Continuity Conditions
- ☐ Geometric Continuity Conditions

a. Parametric Continuity Conditions

- If each section of a spline curve is described with a set of parametric coordinate functions of the form
$$\mathbf{x} = \mathbf{x}(u), \mathbf{y} = \mathbf{y}(u), \mathbf{z} = \mathbf{z}(u), \quad u_1 \leq u \leq u_2$$
- we set **parametric continuity** by matching the parametric derivatives of adjoining curve sections at their common boundary



First Order

b. Geometric Continuity Conditions

- Another method for joining two successive curve sections is to specify conditions for **geometric continuity**
- we require only that the parametric derivatives of the two sections are proportional to each other at their common boundary, instead of requiring equality
- It suffices to require that the directions are the same:



Parametric Cubic Curve

❑ The parametric representation of the curve is as follows:

$$\mathbf{x} = \mathbf{x}(u), \mathbf{y} = \mathbf{y}(u), \mathbf{z} = \mathbf{z}(u), \quad u_1 \leq u \leq u_2$$

❑ A curve is approximated by a **piecewise polynomial curve** instead of piecewise linear curve

❑ Piecewise linear curve is represented by linear and polylines and
Piecewise polynomial curve is represented by a polynomial equation

❑ Cubic polynomial means the polynomials which represent the curve with **degree three**

- ❑ The parametric **cubic polynomial** that is to be fitted between each pair of control points with the following set of equations

$$\begin{aligned}x(u) &= a_x u^3 + b_x u^2 + c_x u + d_x \\y(u) &= a_y u^3 + b_y u^2 + c_y u + d_y, \quad (0 \leq u \leq 1) \\z(u) &= a_z u^3 + b_z u^2 + c_z u + d_z\end{aligned}$$

- ❑ For each of these three equations, we need to determine the values for the four coefficients a , b , c , and d in the polynomial representation for each of the n curve sections between the $n+1$ control points
- ❑ We do this by setting enough boundary conditions at the control-point positions between curve sections so that we can obtain numerical values for all the coefficients

$$\mathbf{P}(u) = (x(u), y(u), z(u)) = \begin{pmatrix} a_x u^3 + b_x u^2 + c_x u + d_x \\ a_y u^3 + b_y u^2 + c_y u + d_y \\ a_z u^3 + b_z u^2 + c_z u + d_z \end{pmatrix}, \text{ with } 0 \leq u \leq 1$$

$$= \begin{pmatrix} u^3 & u^2 & u & 1 \end{pmatrix} \begin{pmatrix} a_x & a_y & a_z \\ b_x & b_y & b_z \\ c_x & c_y & c_z \\ d_x & d_y & d_z \end{pmatrix} = \mathbf{UC}.$$

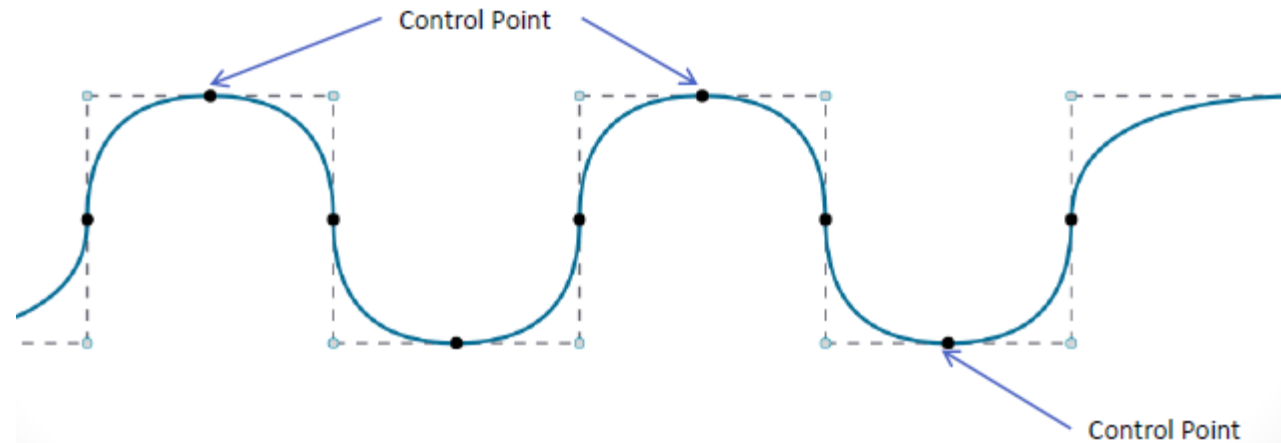
- **U:** Powers of u **C:** Coefficient matrix

Spline Curve

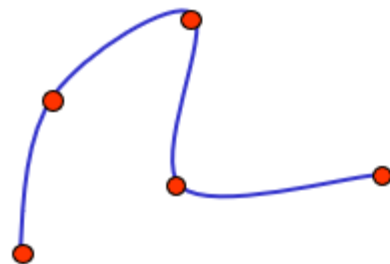
- A "spline curve" in computer graphics is a smooth, continuous curve created by connecting a series of points (called control points) using mathematical functions, allowing for precise control over the shape of a curve while maintaining smoothness, often used to model complex shapes in 3D modeling and design applications;
- Essentially, it's a way to draw a smooth curve through a set of points by dividing the curve into smaller polynomial segments that seamlessly join together



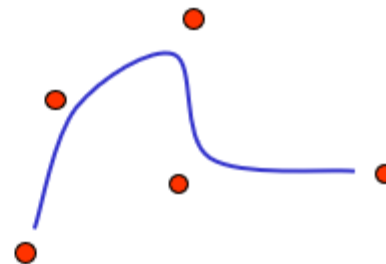
- The general approach is that the user enters a sequence of points and a curve is constructed whose shape closely follows this sequence
- The points are called **control point** which are then fitted with piecewise continuous parametric polynomial functions in one of two ways : interpolation or approximation



1. A curve which actually passes through each control point is called **interpolating curve**
 - Interpolation methods are commonly used to digitize drawings or to specify animation paths
2. A curve that passes near to the control point but not necessarily through them is called an **approximating curve**
 - Approximation methods are used primarily as design tools to create object shapes

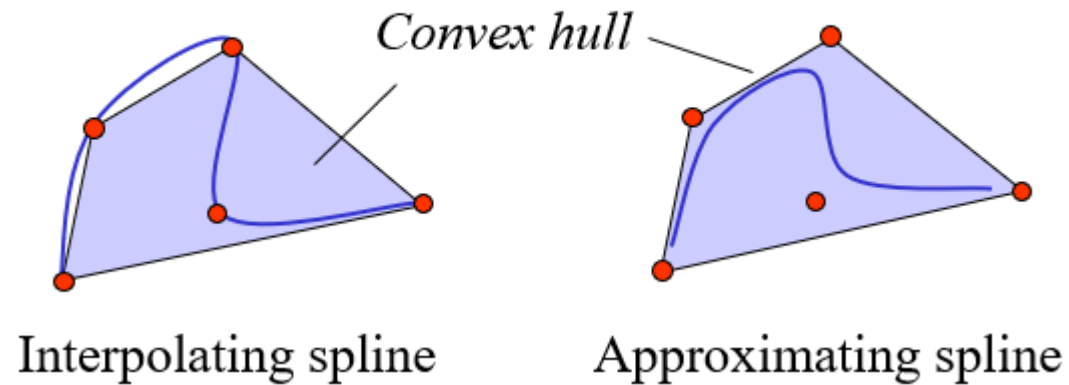


Interpolating spline

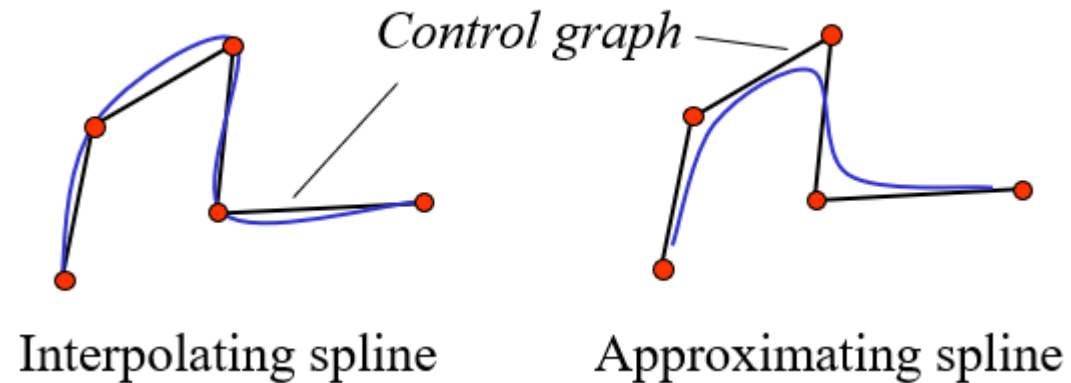


Approximating spline

- A set of control points forms a boundary for a region of space that is called the **convex hull**
- The convex hull for a two-dimensional spline curve is a convex polygon
- In three-dimensional space, the convex hull for a set of spline control points forms a convex polyhedron



- **Control graph:** A polyline connecting the sequence of control points for an approximation spline curve is usually displayed to remind a designer of the control-point positions and ordering



Spline Specifications:

There are three equivalent methods for specifying a particular spline representation:

- a) **Boundary condition:** We can state the set of boundary conditions that are imposed on the spline.

$$x(u) = a_x u^3 + b_x u^2 + c_x u + d_x \quad 0 \leq u \leq 1$$

Boundary condition for this curve can be set for $x(0)$, $x(1)$, $x'(0)$ & $x'(1)$. These four conditions are sufficient to determine the values of four coefficient a_x , b_x , c_x & d_x .

- b) **Characterizing matrix:** We can state the matrix that characterizes the spline.

From the boundary condition, the characterizing matrix for spline is:

$$x(u) = \begin{bmatrix} u^3 & u^2 & u & 1 \end{bmatrix} \begin{bmatrix} a_x \\ b_x \\ c_x \\ d_x \end{bmatrix} \\ = U \cdot C$$

- c) **Blending Function:** We can state the set of blending functions (or basis functions) that determine how specified geometric constraints on the curve are combined to calculate positions along the curve path.

$$x(u) = \sum_{k=0}^3 g_k \cdot BF_k(u)$$

g_k = Geometric constrain parameter

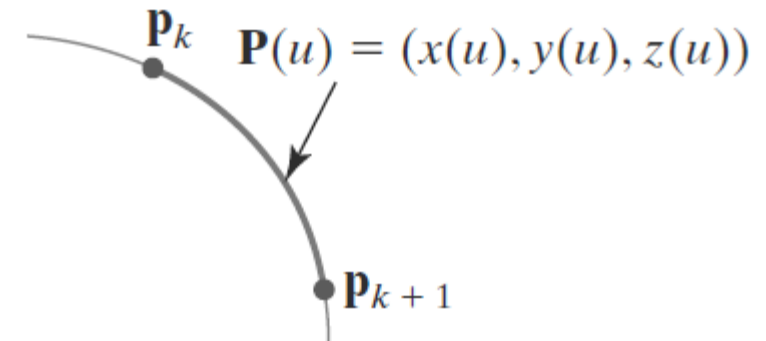
$BF_k(u)$ = Polynomial blending function

Spline Curve Families

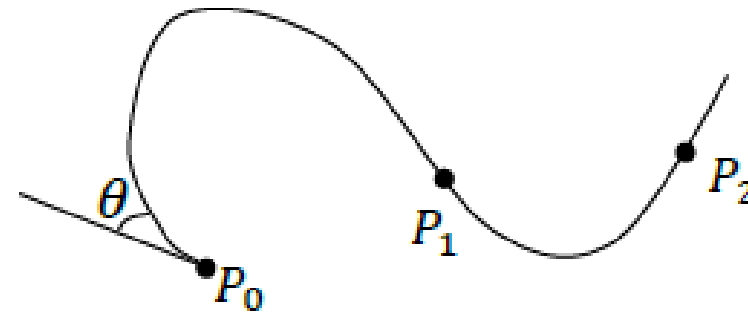
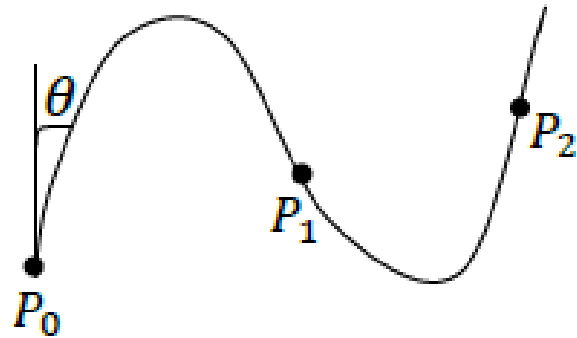
- **Hermite cubic**
 - Defined by its 2 endpoints and tangent vectors at endpoints.
 - Interpolates all its control points.
 - Special case of Bezier and B-Spline.
- **Bezier**
 - Interpolates first and last control points.
 - Curve is tangent to first and last segments of control polygon.
 - Easy to subdivide.
 - Curve segment lies within convex hull of control polygon.
- **B-Spline**
 - Not guaranteed to interpolate control points.
 - Curve segment lies within convex hull of control polygon.
 - Greater local control than Bezier.

1. Hermite Spline

- A **Hermite spline** (named after the French mathematician Charles Hermite) is an **interpolating spline**
- Hermite spline is piecewise cubic polynomial with a specified tangent at each control point
- Hermite splines can be adjusted locally because each curve section depends only on its endpoint constraints



- ❑ It is an interpolating piecewise cubic polynomial with a specified tangent at each control point.
- ❑ If we change the control point at P_0 , then the curve will also change, so that angle θ between P_0 and tangent at P_0 will remain constant.



Hermite Cubic Spline

❑ It has local control over the curve i.e. each curve section depend on its end point only.

❑ The vector equivalence of Hermite curve is

$$P(u) = au^3 + bu^2 + cu + d \dots\dots\dots (i)$$

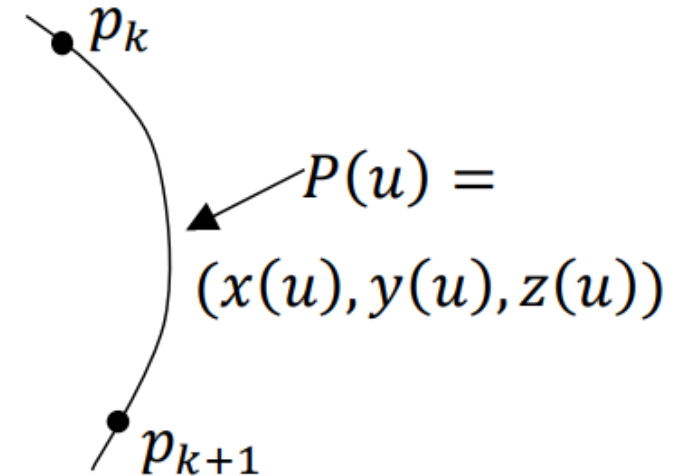
Where, x component of $P(u)$ is

$$x(u) = a_x u^3 + b_x u^2 + c_x u + d_x$$

Similarly, y and z component

$$y(u) = a_y u^3 + b_y u^2 + c_y u + d_y$$

$$z(u) = a_z u^3 + b_z u^2 + c_z u + d_z$$



- Let $P(u)$ denotes the parametric cubic point function for the curve section between control point p_k & p_{k+1} .

At p_k , $u=0$

$$\therefore P(0) = p_k$$

At p_{k+1} , $u=1$

$$\therefore P(1) = p_{k+1}$$

Also let Dp_k & Dp_{k+1} denote the slope at p_k & p_{k+1} .

$$\therefore P'(0) = Dp_k$$

$$P'(1) = Dp_{k+1}$$

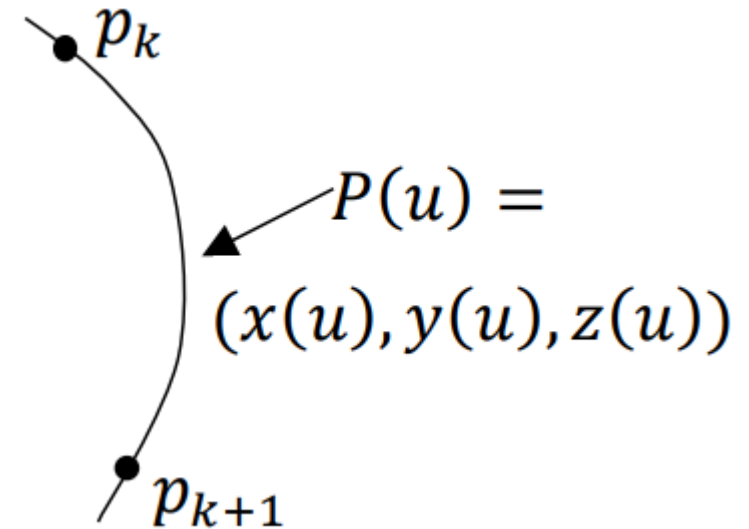
Hence boundary condition for Hermite curve

$$P(0) = p_k$$

$$P(1) = p_{k+1} \quad \dots\dots\dots (ii)$$

$$P'(0) = Dp_k$$

$$P'(1) = Dp_{k+1}$$



Matrix equivalent of eq. (i) is

$$P(u) = [u^3 \quad u^2 \quad u \quad 1] \begin{bmatrix} a \\ b \\ c \\ d \end{bmatrix} \dots\dots\dots (iii)$$

Similarly derivative of point function can be represented as,

$$P'(u) = [3u^2 \quad 2u \quad 1 \quad 0] \begin{bmatrix} a \\ b \\ c \\ d \end{bmatrix}$$

In matrix form, the Hermite boundary condition from eq. (ii) can be represented as

$$\begin{bmatrix} p_k \\ p_{k+1} \\ Dp_k \\ Dp_{k+1} \end{bmatrix} = \begin{bmatrix} 0 & 0 & 0 & 1 \\ 1 & 1 & 1 & 1 \\ 0 & 0 & 1 & 0 \\ 3 & 2 & 1 & 0 \end{bmatrix} \begin{bmatrix} a \\ b \\ c \\ d \end{bmatrix} \dots\dots\dots (iv)$$

Rearrange eqn ii

$$U = [U^3 \ U^2 \ U \ 1]$$

$$P(0) = [0 \ 0 \ 0 \ 1]$$

$$P(1) = [1 \ 1 \ 1 \ 1]$$

$$P'(0) = [3U^2 \ 2U \ 1 \ 0] = [0 \ 0 \ 1 \ 0]$$

$$P'(1) = [3 \ 2 \ 1 \ 0]$$

Solving eq. (iv) for polynomial coefficient, we have

$$\begin{bmatrix} a \\ b \\ c \\ d \end{bmatrix} = \begin{bmatrix} 0 & 0 & 0 & 1 \\ 1 & 1 & 1 & 1 \\ 0 & 0 & 1 & 0 \\ 3 & 2 & 1 & 0 \end{bmatrix}^{-1} \cdot \begin{bmatrix} p_k \\ p_{k+1} \\ Dp_k \\ Dp_{k+1} \end{bmatrix} = \begin{bmatrix} 2 & -2 & 1 & 1 \\ -3 & 3 & -2 & -1 \\ 0 & 0 & 1 & 0 \\ 1 & 0 & 0 & 0 \end{bmatrix} \cdot \begin{bmatrix} p_k \\ p_{k+1} \\ Dp_k \\ Dp_{k+1} \end{bmatrix} = M_H \cdot \begin{bmatrix} p_k \\ p_{k+1} \\ Dp_k \\ Dp_{k+1} \end{bmatrix}$$

Where, M_H is the Hermite matrix.

Hence eq. (iii) can be represented as

$$P(u) = [u^3 \quad u^2 \quad u \quad 1] \cdot M_H \cdot \begin{bmatrix} p_k \\ p_{k+1} \\ Dp_k \\ Dp_{k+1} \end{bmatrix} \dots\dots\dots (v)$$

Expanding (v)

$$P(u) = p_k(2u^3 - 3u^2 + 1) + p_{k+1}(-2u^3 + 3u^2) + Dp_k(u^3 - 2u^2 + u) + Dp_{k+1}(u^3 - u^2)$$

In terms of Hermite blending function, 'H', the Hermite curve can be represented as:

$$\boxed{P(u) = p_k H_0(u) + p_{k+1} H_1(u) + Dp_k H_2(u) + Dp_{k+1} H_3(u)}$$

Hermite Cubic Spline

The Hermite curves have **several advantages**:

- **Smoothness:**

- Easier to join at endpoints.
- Control over derivatives (directions) at endpoints ensures smooth transitions between joined curves.
- Ensuring the second curve starts with the same derivative the first one ends with guarantees smoothness.

- **Blending Functions:**

- No zeros in the interval.
- The influence of control points remains positive and does not switch to negative.
- The influence of the first control point peaks at the beginning and steadily drops to zero.

- **Ease of Control:**

- No need to know interior points.
- Useful when only the start and end of the curve are known.

❑ Displaying Hermite curves

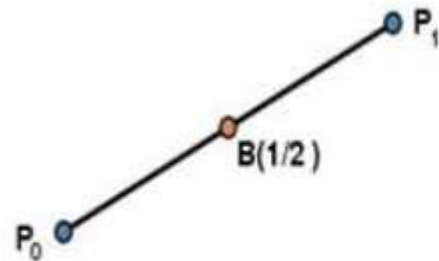
- ❑ Evaluate the curve at a fixed set of parameter values and join the points with straight lines.

❑ From Hermite to Bezier curves

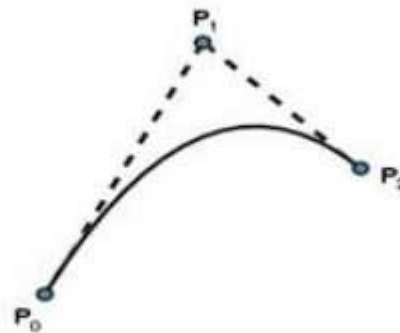
- ❑ Hermite curves are difficult to use because we usually have control points but not derivatives.
- ❑ However, Hermite curves are the basis of the Bezier curves.
- ❑ Bezier curves are more intuitive, since we only need to specify control points.

Bézier Curves

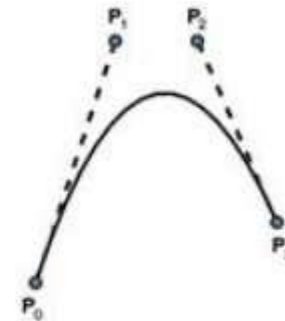
- Bezier curve is discovered by the French engineer Pierre Bézier.
- Bezier curve is an *approximation* of given control points
- Approximate tangents by using control points are used to generate curve.
- Bezier curve of degree n is defined over $n+1$ control points $\{P_i\}_{i=0,\dots,n}$.



Simple Bezier Curve



Quadratic Bezier Curve



Cubic Bezier Curve

The **Bezier curve** has **two important** properties:

1. It always passes through the first and last control points.
2. It lies within the convex hull (convex polynomial boundary) of the control points. This follows from the properties of Bezier blending function: they are positive and their sum is always 1, i.e.

$$\sum_{k=0}^n \text{BEZ}_{k,n}(u) = 1$$

General Bezier curve for (n+1) control point, denoted as $p_k = (x_k, y_k, z_k)$ with 'k' varying from 0 to n is given by

$$P(u) = \sum_{k=0}^n p_k BEZ_{k,n}(u), \quad 0 \leq u \leq 1 \quad \dots \dots \dots (i)$$

Where, $P(u)$ is a point on Bezier Curve.

p_k is a control point.

$BEZ_{k,n}(u)$ is a Bezier blending function also known as *Bernstein Polynomial*.

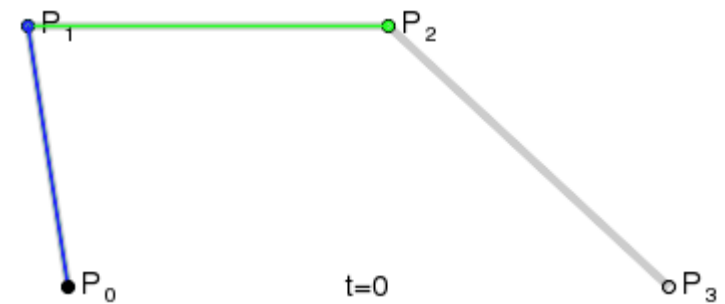
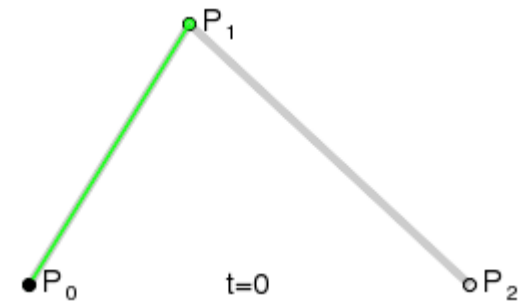
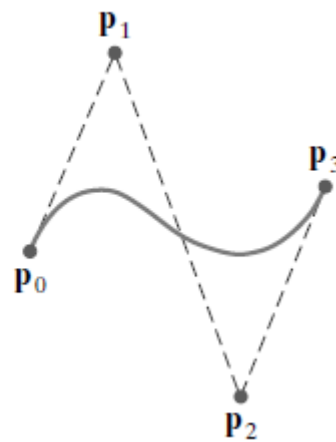
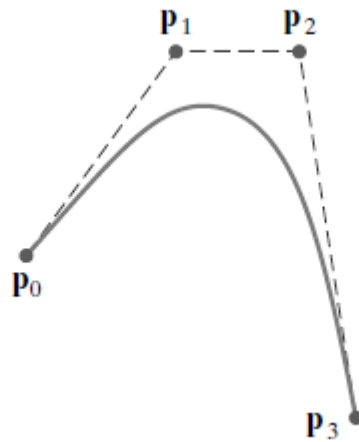
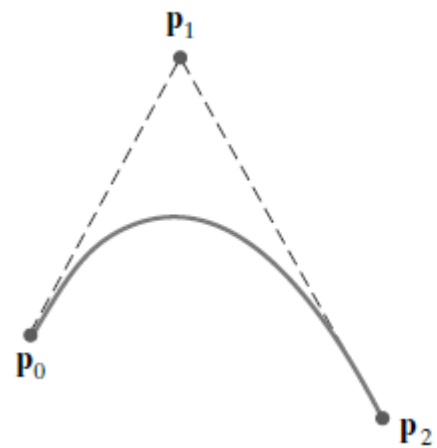
Bezier blending function is defined as

$$BEZ_{k,n}(u) = C(n, k) u^k (1 - u)^{n-k}$$

Where, $C(n, k) = \frac{n!}{k!(n-k)!}$

Individual x, y, z coordinates an a Bezier curve is given by,

$$x(u) = \sum_{k=0}^n x_k BEZ_{k,n}(u) \quad y(u) = \sum_{k=0}^n y_k BEZ_{k,n}(u) \quad z(u) = \sum_{k=0}^n z_k BEZ_{k,n}(u)$$



Examples of two-dimensional Bézier curves generated with three, four, and five control points. Dashed lines connect the control-point positions

Properties of Bezier curve:

- a) The basis functions are real.
- b) The Bezier curve always passes through first and last control point i.e. $p(0) = p_0$ & $p(1) = p_n$.
- c) The degree of polynomial representing Bezier curve is one less than the number of control points.
- d) The Bezier curve always follows convex hull formed by control points.
- e) The Bezier curve always lies inside the polygon formed by control points.
- f) Bezier blending functions are positive and sum is equal to 1.
$$\sum_{k=0}^n BEZ_{k,n}(u) = 1$$
- g) The direction of the tangent vector at the end points is same like vector determined by first and last segment.

Cubic Bezier Curve

- It is a Bezier curve generated by four control points.
- General equation for cubic Bezier curve is

$$P(u) = \sum_{k=0}^3 p_k BEZ_{k,n}(u), \quad 0 \leq u \leq 1 \quad \dots \dots \dots (i)$$

$$P(u) = p_0 BEZ_{0,3}(u) + p_1 BEZ_{1,3}(u) + p_2 BEZ_{2,3}(u) + p_3 BEZ_{3,3}(u)$$

Where,

$$BEZ_{0,3}(u) = C(3,0)u^0(1-u)^{3-0} = \frac{3!}{0!(3-0)!} \times (1-u)^3 = (1-u)^3$$

Similarly,

$$BEZ_{1,3}(u) = 3u(1-u)^2$$

$$BEZ_{2,3}(u) = 3u^2(1-u)$$

$$BEZ_{3,3}(u) = u^3$$

$$\therefore P(u) = p_0(1-u)^3 + p_1 3u(1-u)^2 + p_2 3u^2(1-u) + p_3 u^3$$

$$\begin{aligned}
 BEZ_{0,3} &= (1-u)^3 \\
 &= 1-3.1^2.u+3.1.u^2-u^3 \\
 &= -u^3+3u^2-3u+1 \\
 &= [-1 \ 3 \ -3 \ 1]
 \end{aligned}$$

$$\begin{aligned}
 BEZ_{2,3} &= 3u^2(1-u) \\
 &= -3u^3+3u^2 \\
 &= [-3 \ 3 \ 0 \ 0]
 \end{aligned}$$

$$\begin{aligned}
 BEZ_{1,3} &= 3u(1-u)^2 \\
 &= 3u(1-2u+u^2) \\
 &= 3u^3-6u^2+3u \\
 &= [3 \ -6 \ 3 \ 0]
 \end{aligned}$$

$$\begin{aligned}
 BEZ_{3,3} &= u^3 \\
 &= [1 \ 0 \ 0 \ 0]
 \end{aligned}$$

In Matrix form

$$P(u) = [u^3 \quad u^2 \quad u \quad 1] \cdot M_{BEZ} \cdot \begin{bmatrix} p_0 \\ p_1 \\ p_2 \\ p_3 \end{bmatrix}$$

Where,

$$M_{BEZ} = \text{Bezier matrix} = \begin{bmatrix} -1 & 3 & -3 & 1 \\ 3 & -6 & 3 & 0 \\ -3 & 3 & 0 & 0 \\ 1 & 0 & 0 & 0 \end{bmatrix}$$

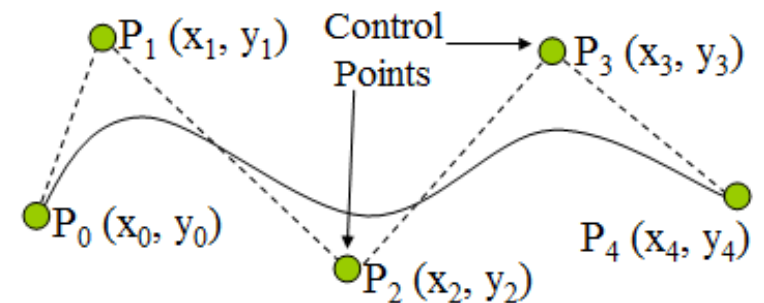


Fig. Bezier curve with 4-control points in xy-plane (z=0)

Numerical : Construct Bezier curve for control points (4, 2), (8, 8) and (16, 4)

Solution:

Given control points

$$p_0 = (x_0, y_0) = (4, 2)$$

$$p_1 = (x_1, y_1) = (8, 8)$$

$$p_2 = (x_2, y_2) = (16, 4)$$

Here, degree (or order) $n = 2$

We have basis function as

$$P(u) = \sum_{k=0}^n p_k BEZ_{k,n}(u), \quad 0 \leq u \leq 1$$

$$= \sum_{k=0}^2 p_k BEZ_{k,2}(u), \quad 0 \leq u \leq 1$$

$$\therefore P(u) = p_0 BEZ_{0,2}(u) + p_1 BEZ_{1,2}(u) + p_2 BEZ_{2,2}(u)$$

Parametric equations are;

$$x(u) = x_0 BEZ_{0,2}(u) + x_1 BEZ_{1,2}(u) + x_2 BEZ_{2,2}(u) \dots\dots\dots (i)$$

$$y(u) = y_0 BEZ_{0,2}(u) + y_1 BEZ_{1,2}(u) + y_2 BEZ_{2,2}(u) \dots\dots\dots (ii)$$

Now,

$$BEZ_{0,2}(u) = C(2,0)u^0(1-u)^{2-0} = \frac{2!}{0!(2-0)!} \times (1-u)^2 = (1-u)^2$$

$$BEZ_{1,2}(u) = C(2,1)u^1(1-u)^{2-1} = \frac{2!}{1!(2-1)!} \times u(1-u)^1 = 2u(1-u)$$

$$BEZ_{2,2}(u) = C(2,2)u^2(1-u)^{2-2} = \frac{2!}{2!(2-2)!} \times u^2(1-u)^0 = u^2$$

Putting these values in eq. (i) & (ii) we get;

$$x(u) = x_0(1-u)^2 + x_1 2u(1-u) + x_2 u^2 = 4u^2 + 8u + 4$$

$$y(u) = y_0(1-u)^2 + y_1 2u(1-u) + y_2 u^2 = -10u^2 + 12u + 2$$

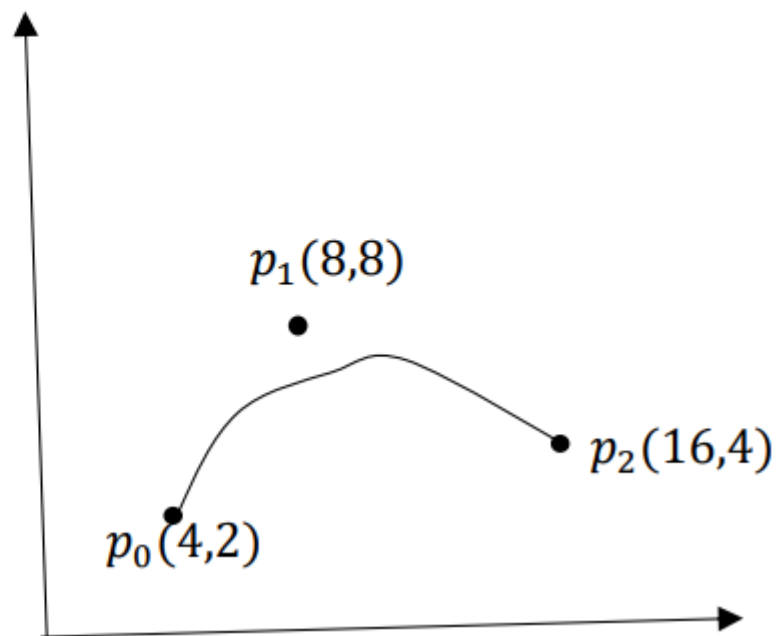
$$x(u) = 4u^2 + 8u + 4$$

$$y(u) = -10u^2 + 12u + 2$$

Now,

	$x(u)$	$y(u)$
$u=0$	4	2
$u=0.2$	5.76	4.0
$u=0.4$	7.84	5.20
$u=0.6$	10.24	5.6
$u=0.8$	12.96	5.2
$u=1$	16	4

Drawing these points we get:



Construct the **Bezier curve of order 3** with 4 vertices of the control polygon **$p_0(0, 0)$, $p_1(1, 2)$, $p_2(3, 2)$ & $p_3(2, 0)$** . Generate at least 5 points on the curve.

Solution:

Given control points

$$p_0 = (x_0, y_0) = (0, 0)$$

$$p_1 = (x_1, y_1) = (1, 2)$$

$$p_2 = (x_2, y_2) = (3, 2)$$

$$p_3 = (x_3, y_3) = (2, 0)$$

Here, degree $n = 3$

We have basis function as

$$\begin{aligned} P(u) &= \sum_{k=0}^n p_k BEZ_{k,n}(u), & 0 \leq u \leq 1 \\ &= \sum_{k=0}^3 p_k BEZ_{k,3}(u), & 0 \leq u \leq 1 \end{aligned}$$

$$\therefore P(u) = p_0 BEZ_{0,3}(u) + p_1 BEZ_{1,3}(u) + p_2 BEZ_{2,3}(u) + p_3 BEZ_{3,3}(u)$$

Parametric equations are;

$$x(u) = x_0 BEZ_{0,3}(u) + x_1 BEZ_{1,3}(u) + x_2 BEZ_{2,3}(u) + x_3 BEZ_{3,3}(u) \dots\dots\dots (i)$$

$$y(u) = y_0 BEZ_{0,3}(u) + y_1 BEZ_{1,3}(u) + y_2 BEZ_{2,3}(u) + y_3 BEZ_{3,3}(u) \dots\dots\dots (ii)$$

Now,

$$BEZ_{0,3}(u) = C(3,0)u^0(1-u)^{3-0} = \frac{3!}{0!(3-0)!} \times (1-u)^3 = (1-u)^3$$

$$BEZ_{1,3}(u) = C(3,1)u^1(1-u)^{3-1} = \frac{3!}{1!(3-1)!} \times u(1-u)^2 = 3u(1-u)^2$$

$$BEZ_{2,3}(u) = C(3,2)u^2(1-u)^{3-2} = \frac{3!}{2!(3-2)!} \times u^2(1-u)^1 = 3u^2(1-u)$$

$$BEZ_{3,3}(u) = C(3,3)u^3(1-u)^{3-3} = \frac{3!}{3!(3-3)!} \times u^3(1-u)^0 = u^3$$

Putting these values in eq. (i) & (ii) we get;

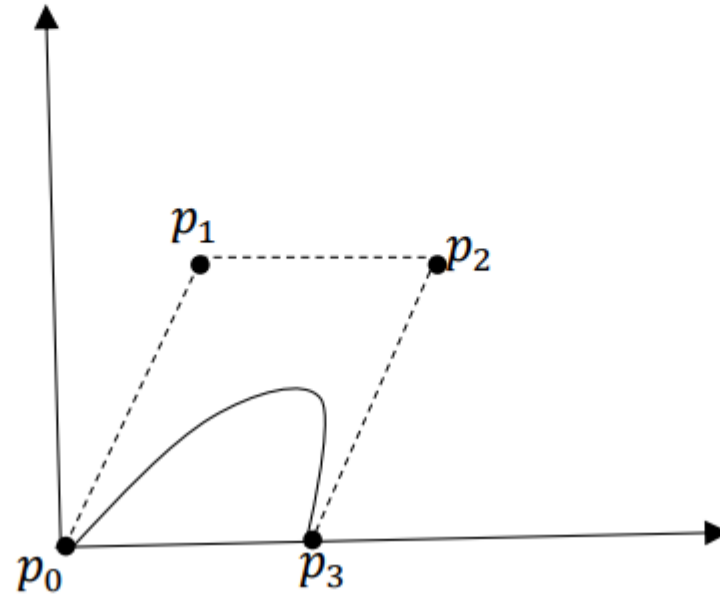
$$x(u) = x_0(1-u)^3 + x_1 3u(1-u)^2 + x_2 3u^2(1-u) + x_3 u^3 = -4u^3 + 3u^2 + 3u$$

$$y(u) = y_0(1-u)^3 + y_1 3u(1-u)^2 + y_2 3u^2(1-u) + y_3 u^3 = -6u^2 + 6u$$

Now,

	$x(u)$	$y(u)$
$u=0$	0	0
$u=0.15$	0.50	0.76
$u=0.35$	1.24	1.36
$u=0.5$	1.75	1.50
$u=0.65$	2.12	1.36
$u=0.85$	2.14	0.76
$u=1$	2	0

Plotting the graph;



Classwork

Q.N. > Calculate (x,y) coordinates of Bézier curve described by the following 4 control points: (0,0), (1,2), (3,3), (4,0).

Q.N. Calculate (x,y) coordinates of Bézier curve described by the following 4 control points: (0,0), (1,2), (3,3), (4,0).

Step by step solution

For four control points, $n = 3$.

First calculate all the blending functions, B_{kn} for $k=0,..,n$ using the formula:

$$B_{kn}(u) = C(n, k) u^k (1 - u)^{n-k} = \frac{n!}{k! \cdot (n-k)!} u^k (1 - u)^{n-k}$$

$$B_{03}(u) = \frac{3!}{0! \cdot 3!} u^0 (1 - u)^3 = 1 \cdot u^0 (1 - u)^3 = (1 - u)^3$$

$$B_{13}(u) = \frac{3!}{1! \cdot 2!} u^1 (1 - u)^2 = 3 \cdot u^1 (1 - u)^2 = 3u \cdot (1 - u)^2$$

$$B_{23}(u) = \frac{3!}{2! \cdot 1!} u^2 (1 - u)^1 = 3 \cdot u^2 (1 - u)^1 = 3u^2 (1 - u)$$

$$B_{33}(u) = \frac{3!}{3! \cdot 0!} u^3 (1 - u)^0 = 1 \cdot u^3 (1 - u)^0 = u^3$$

Q.N. Calculate (x,y) coordinates of Bézier curve described by the following 4 control points: (0,0), (1,2), (3,3), (4,0).

$$\begin{aligned}\underline{u=0.0} \ x(0) &= \sum_{k=0}^n x_k B_{k,n}(0) = x_0 B_{0,3}(0) + x_1 B_{1,3}(0) + x_2 B_{2,3}(0) + x_3 B_{3,3}(0) = \\ &= 0 \cdot (1-u)^3 + 1 \cdot 3u \cdot (1-u)^2 + 3 \cdot 3u^2 (1-u) + 4 \cdot u^3 = \\ &= 0 \cdot 1 + 1 \cdot 0 + 3 \cdot 0 + 4 \cdot 0 = \\ &= 0\end{aligned}$$

$$\begin{aligned}y(0) &= \sum_{k=0}^n y_k B_{k,n}(0) = y_0 B_{0,3}(0) + y_1 B_{1,3}(0) + y_2 B_{2,3}(0) + y_3 B_{3,3}(0) = \\ &= 0 \cdot (1-u)^3 + 2 \cdot 3u \cdot (1-u)^2 + 3 \cdot 3u^2 (1-u) + 0 \cdot u^3 = \\ &= 0\end{aligned}$$

Q.N. Calculate (x,y) coordinates of Bézier curve described by the following 4 control points: (0,0), (1,2), (3,3), (4,0).

$$\begin{aligned}\underline{u = 0.2} \quad x(0.2) &= \sum_{k=0}^n x_k B_{kn}(0.2) = x_0 B_{03}(0.2) + x_1 B_{13}(0.2) + x_2 B_{23}(0.2) + x_3 B_{33}(0.2) = \\ &= 0 \cdot (1 - u)^3 + 1 \cdot 3u \cdot (1 - u)^2 + 3 \cdot 3u^2 (1 - u) + 4 \cdot u^3 = \\ &= 0 \cdot 0.512 + 1 \cdot 0.384 + 3 \cdot 0.096 + 4 \cdot 0.008 = \\ &= 0.7\end{aligned}$$

$$\begin{aligned}y(0.2) &= \sum_{k=0}^n y_k B_{kn}(0.2) = y_0 B_{03}(0.2) + y_1 B_{13}(0.2) + y_2 B_{23}(0.2) + y_3 B_{33}(0.2) = \\ &= 0 \cdot (1 - u)^3 + 2 \cdot 3u \cdot (1 - u)^2 + 3 \cdot 3u^2 (1 - u) + 0 \cdot u^3 = \\ &= 0 \cdot 0.512 + 2 \cdot 0.384 + 3 \cdot 0.096 + 0 \cdot 0.008 = \\ &= 1.1\end{aligned}$$

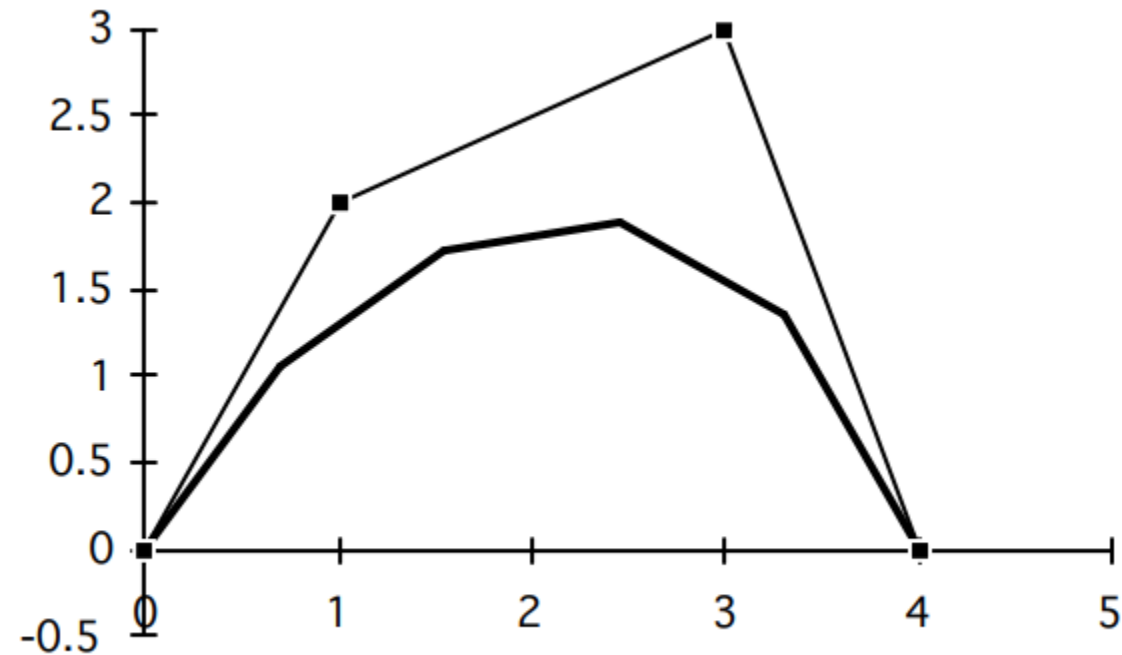
Q.N. Calculate (x,y) coordinates of Bézier curve described by the following 4 control points: (0,0), (1,2), (3,3), (4,0).

etc, giving:

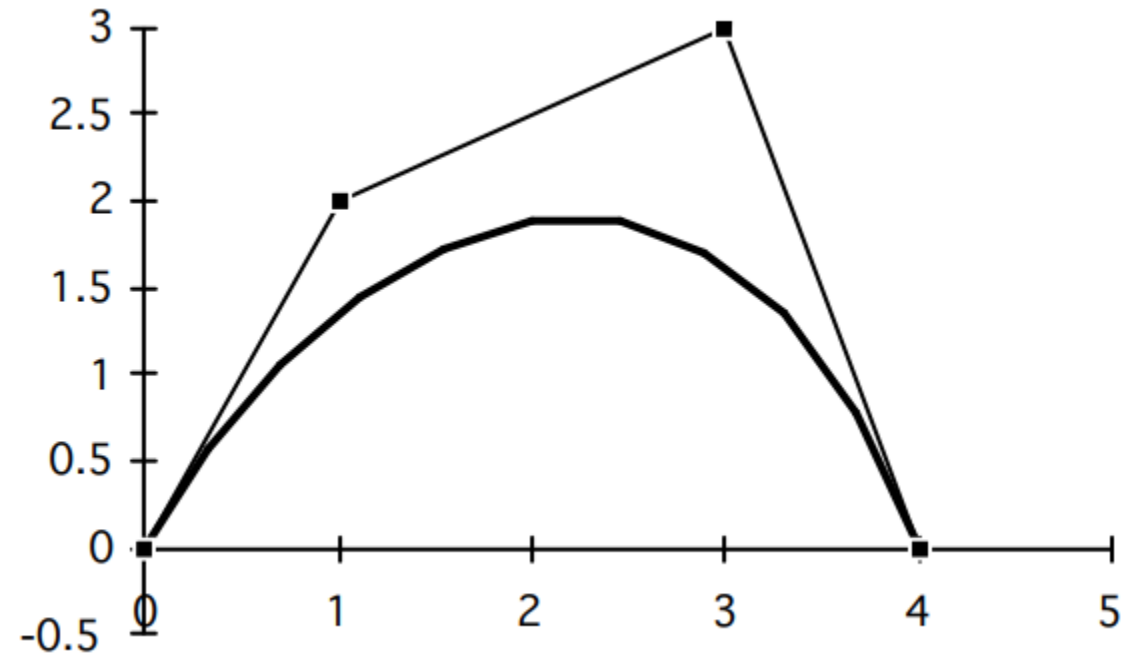
$u = 0.0$	$x(u) = 0.0$	$y(u) = 0.0$
$u = 0.2$	$x(u) = 0.7$	$y(u) = 1.1$
$u = 0.4$	$x(u) = 1.55$	$y(u) = 1.7$
$u = 0.6$	$x(u) = 2.45$	$y(u) = 1.9$
$u = 0.8$	$x(u) = 3.3$	$y(u) = 1.3$
$u = 1.0$	$x(u) = 4.0$	$y(u) = 0.0$

$(x(u), y(u))_{u=0,1}$ are coordinates of the curve points.

The plot below shows control points (joined with a thin line) and a Bézier curve with 6 steps



The plot below shows control points (joined with a thin line) and a Bézier curve with 11 steps; note smoother appearance of this curve in comparison to the previous one.



Classwork

1. Given control points (10,100), (50, 100), (70,120) and (100, 150). Calculate coordinates of any four points lying on the corresponding Beizer curve.
2. Set up the equation of Beizer curve and roughly trace it for three control points (1,1), (2,2) and (3,1).
3. Construct the Bezier curve of order 3 and 4 polygon vertices A(1,1) B(2,3) C(4,3) and D(6,4).
4. Construct the Bezier curve of order 3 with 4 vertices of the control polygon $p_0(0, 0)$, $p_1(1, 2)$, $p_2(3, 2)$ & $p_3(2, 0)$. Generate at least 5 points on the curve.

B Spline Curve(Assignment)

- ❑ Both B-spline and Bezier curves are parametric, meaning they are defined by a set of control points and a parameter.
- ❑ In Bezier curves, altering any control point affects the entire curve shape. B-spline curves provide local control, so changing a control point only affects a specific segment of the curve.
- ❑ B-spline curves offer greater flexibility and control over the curve shape due to the local influence of each control point.
- ❑ B-spline curves can be of any degree, and the number of control points can be adjusted without changing the degree of the curve.

- This spline category is the most widely used, and **B-spline** functions are commonly available in CAD systems and many graphics-programming packages.
- Like Bézier splines, B-splines are generated by approximating a set of control points.

But **B-splines** have two advantages over Bézier splines:

- (1) the degree of a B-spline polynomial can be set independently of the number of control points (with certain limitations)
- (2) B-splines allow local control over the shape of a spline

The tradeoff is that B-splines are more complex than Bézier splines

□ General equation of B-spline curve is given by

$$P(u) = \sum_{k=0}^n p_k B_{k,d}(u), \quad u_{\min} \leq u \leq u_{\max}, \quad 2 \leq d \leq n+1$$

□ Where, p_k is a set of $(n+1)$ control points

$B_{k,d}(u)$ is the B-spline blending function are polynomials of degree $d-1$, where d is the **degree parameter**.

B Spline Curve

- Blending functions for B-spline curves are defined by the Cox-deBoor recursion formulas:

$$B_{k,1}(u) = \begin{cases} 1, & \text{if } u_k \leq u < u_{k+1} \\ 0, & \text{Otherwise} \end{cases}$$

$$B_{k,d}(u) = \frac{u - u_k}{u_{k+d-1} - u_k} B_{k,d-1}(u) + \frac{u_{k+d} - u}{u_{k+d} - u_{k+1}} B_{k+1,d-1}(u)$$

- where each blending function is defined over d subintervals of the total range of u .
- Each subinterval endpoint u_j is referred to as a **knot**, and the entire set of selected subinterval endpoints is called a **knot vector**.

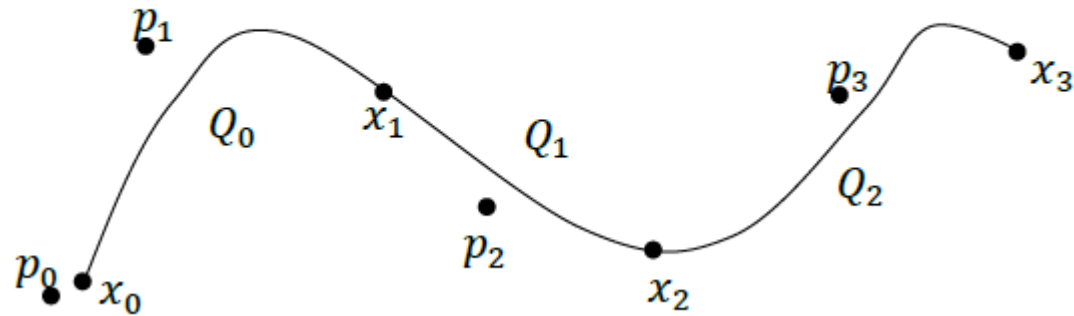
Properties of B-Spline curve

- a) The polynomial curve has $d - 1$ degree
- b) For $(n+1)$ control points, the curve is described with $(n+1)$ blending function
- c) Each blending function $B_{k,d}$ is defined over 'd' sub-interval of the total range of 'u', starting at knot value u_k
- d) The sum of B-spline basis functions for any parameter value is 1

$$\sum_{k=0}^n B_{k,d}(u) = 1$$

- e) Each basis function is positive or zero for all parameter value.
- f) The range of parameter 'u' is divided into $(n+d)$ sub interval by $(n+d+1)$ values specified in knot vector.
- g) Each section of spline curve is influenced by 'd' control point.
- h) Any one control point can affect the shape of at most 'd' curve sections.
- i) The maximum order of curve is equal to the number of vertices of defining polygon.
- j) The curve generally follows the shape of defining polygon.
- k) The degree of B-spline polynomial is independent on the number of vertices of defining polygon.

- p_0, p_1, p_2, p_3 Control point
- x_0, x_1, x_2, x_3 Knot values
- Q_0, Q_1, Q_2 Curve segment
- The knots produce a vector that defines the domain of the curve



B Spline Curve

4.2 Surface Modeling

- ❑ In computer graphics, modeling refers to the process of creating a mathematical representation of a three-dimensional (3D) object
- ❑ Modeling is the fundamental step in creating 3D graphics, forming the basis for all subsequent processes like texturing, lighting, and animation
- ❑ Techniques such as texturing and shading are applied to models to enhance their visual appearance and realism
- ❑ Many kinds of objects in graphics scene. E.g. flowers, trees, clouds, rocks, bricks, marble, steel etc
- ❑ All characteristics can't be described with single way of representation of objects

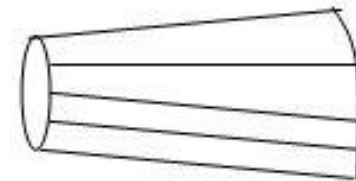
- Two broad categories to represent **Euclidean** (geometric) solid objects:
 - **Boundary Representation (B-reps)**
 - **Space-Partitioning Representations**
- **Natural objects** (Non-Euclidean i.e. non-geometrical shape)
 - Represented by **Fractal Geometry**

3D Object Representation

3D object representation is divided into **two** categories:

a) Boundary Representations (B-reps):

- ❑ B-reps describe a three dimensional object as a set of surface that separates the object interior from the environment
- ❑ B-reps describe the objects exterior. It describes a 3d object as a set of surfaces that encloses the objects interior
- ❑ B-reps for single polyhedron satisfy Euler's formula: $V-E+F=2$
- ❑ **Examples:** *Polygon surfaces and spline patches*, Polyhedron, ellipsoid, aircraft, medical images etc.

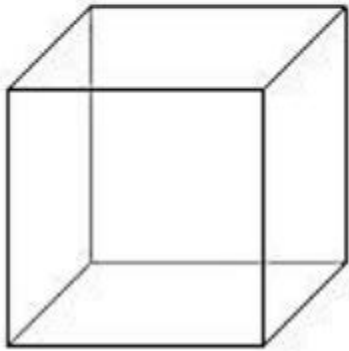


A 3D object represented by polygons

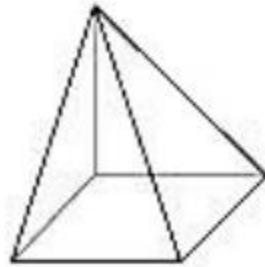
Polyhedron

Solid delimited by a set of polygons whose edges belonging to two polygons (for solids *2-manifolds*).

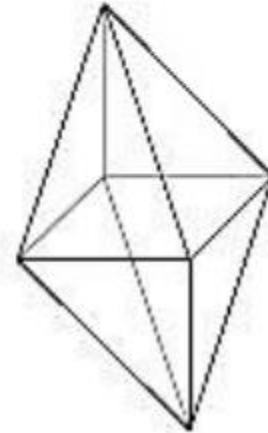
- Boundary representation for single polyhedron satisfy Euler's formula
 $V - E + F = 2$ where, V- vertex, E –edges & F- faces.



$$\begin{aligned} V &= 8 \\ E &= 12 \\ F &= 6 \end{aligned}$$



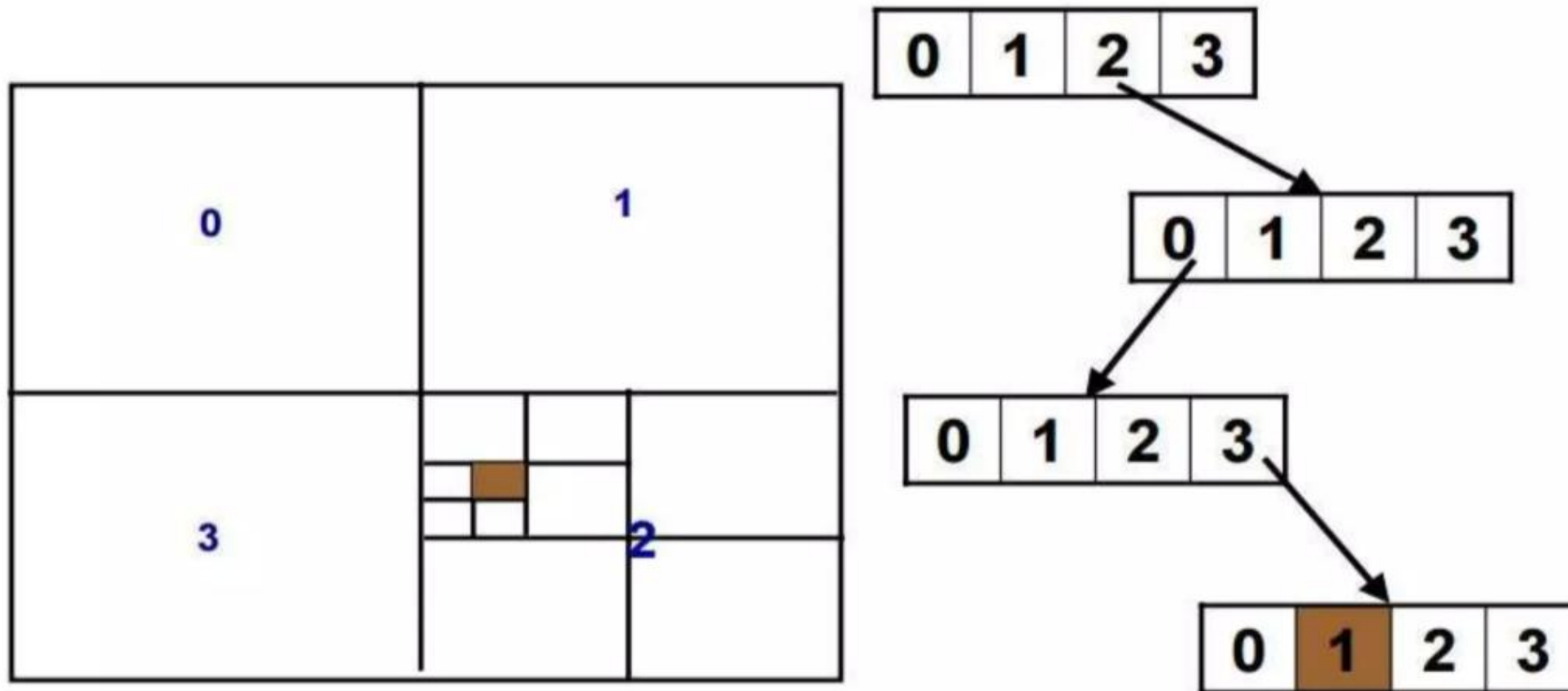
$$\begin{aligned} V &= 5 \\ E &= 8 \\ F &= 5 \end{aligned}$$



$$\begin{aligned} V &= 6 \\ E &= 12 \\ F &= 8 \end{aligned}$$

b) Space Partitioning Representation: Space partitioning describes interior properties by partitioning the spatial region containing an object into a small, non- overlapping, contiguous solids.

Let us visualize 2D Quadtree representation of an image or view.



3D Object Representation

a) Boundary Representations (B-reps)

1. Polygon Surface
2. Quadratic Surface
3. Wireframe representation
4. Blobby objects
5. Spline representation

b) Space partitioning representation

1. Octree representation

1. Polygon Surface

- ❑ The most commonly used boundary representation for a three-dimensional graphics object is a set of surface polygons that enclose the object interior.
- ❑ Many graphics systems store all object descriptions as sets of surface polygons.
- ❑ This simplifies and speeds up the surface rendering and display of objects, since all surfaces are described with linear equations.
- ❑ For this reason, polygon descriptions are often referred to as **"standard graphics objects"**.

- ❑ Curved surface approximation can be improved with increase in number of polygons
- ❑ They are common in design and solid-modeling applications, since wireframe display can be done quickly to give general indication of surface structure.
- ❑ Realistic scenes are produced by interpolating shading patterns across polygon surface to illuminate

Polygon Surface

- Polygon surface can be represented by:

a) Polygon Table

- I. Geometric table
 - i. vertex table
 - ii. edge table
 - iii. polygon surface table
- II. Attribute table

b) Polygon Meshes

c) Plane equation

1.a) Polygon Tables

- ❑ Polygons tables can be used to specify polygon surfaces
- ❑ Specify a polygon surface with a set of vertex coordinates and associated attribute parameters
- ❑ As information for each polygon is input, the data are placed into tables that are to be used in the subsequent processing, display, and manipulation of the objects in a scene

Polygon tables can be organized into two groups:

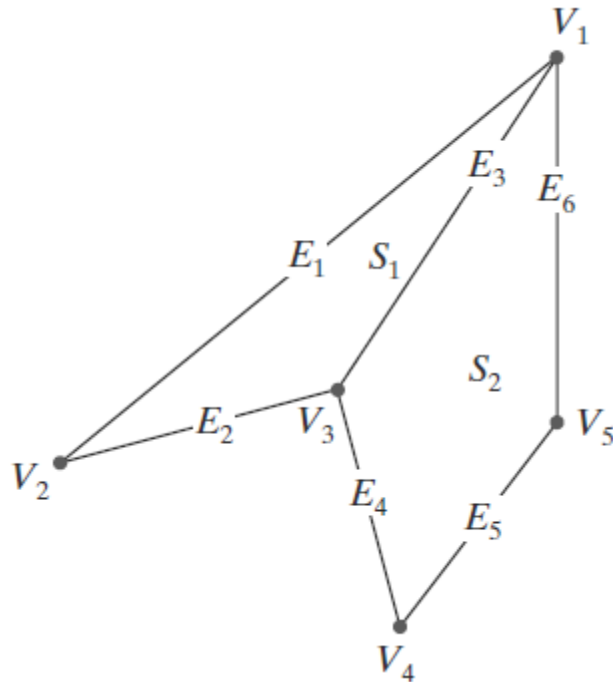
- Geometric tables**
- Attribute tables**

i. Geometric tables

- ❑ Contain vertex coordinates and parameters to identify the spatial orientation of the polygon surfaces
 - ❑ Geometric data consists of three tables: a vertex table, an edge table, and a surface table.
-
- **Vertex table:** It stores co-ordinate values for each vertex of the object
 - **Edge Table:** The edge table contains pointers back into the vertex table to identify the vertices for each polygon edge
 - **Surface table:** And the polygon table contains pointers back into the edge table to identify the edges for each polygon surfaces

i. Geometric tables:

The object can be displayed efficiently by using data from tables and processing them for surface rendering and visible surface determination.



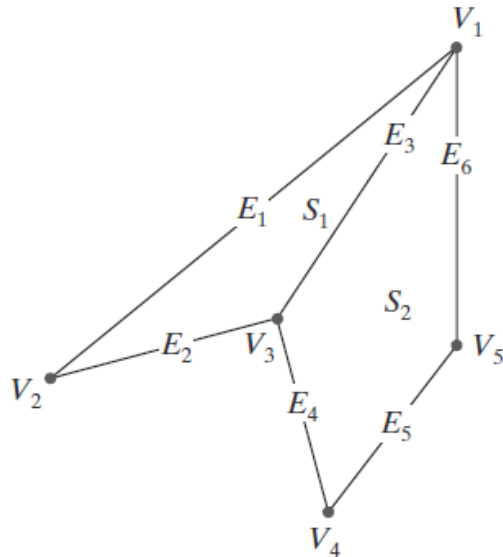
VERTEX TABLE	
V_1 :	x_1, y_1, z_1
V_2 :	x_2, y_2, z_2
V_3 :	x_3, y_3, z_3
V_4 :	x_4, y_4, z_4
V_5 :	x_5, y_5, z_5

EDGE TABLE	
E_1 :	V_1, V_2
E_2 :	V_2, V_3
E_3 :	V_3, V_1
E_4 :	V_3, V_4
E_5 :	V_4, V_5
E_6 :	V_5, V_1

SURFACE-FACET TABLE	
S_1 :	E_1, E_2, E_3
S_2 :	E_3, E_4, E_5, E_6

i. Geometric tables: using forward pointer in edge table

- ❑ add extra information for quick identification of redundancy
- ❑ pointers to polygon table are added to edge table so as to notice common edges quickly



VERTEX TABLE

$v_1: x_1, y_1, z_1$
 $v_2: x_2, y_2, z_2$
 $v_3: x_3, y_3, z_3$
 $v_4: x_4, y_4, z_4$
 $v_5: x_5, y_5, z_5$

EDGE TABLE

$E_1: v_1, v_2, S_1$
 $E_2: v_2, v_3, S_1$
 $E_3: v_3, v_1, S_1, S_2$
 $E_4: v_3, v_4, S_2$
 $E_5: v_4, v_5, S_2$
 $E_6: v_5, v_1, S_2$

SURFACE-FACET TABLE

$S_1: E_1, E_2, E_3$
 $S_2: E_3, E_4, E_5, E_6$

ii) Attribute tables

- Attribute information for an object includes parameters specifying the degree of transparency of the object and its surface reflectivity and texture characteristics.
- The above three table also include the polygon attribute according to their pointer information.

b) Polygon Meshes

- ❑ It is a collection of vertices, edges and faces that define the shape and surface of a 3D object.
- ❑ It is often used in computer games, animation, virtual reality, and computer-aided design (CAD).
- ❑ Some graphics packages (PHIGS programmer's Hierarchical Interactive Graphics Standard) provide several polygon functions for modeling objects.
- **Triangular Mesh**
 - Given n vertices co-ordinates, $n-2$ triangles are tiled to produce strip
- **Quadrilateral Mesh**
 - Given n by m array of vertices co-ordinates, $(n-1)$ by $(m-1)$ quadrilaterals produce mesh

3D Object Representation

b) Polygon Meshes

- Representation Polygon meshes by vertex coordinates

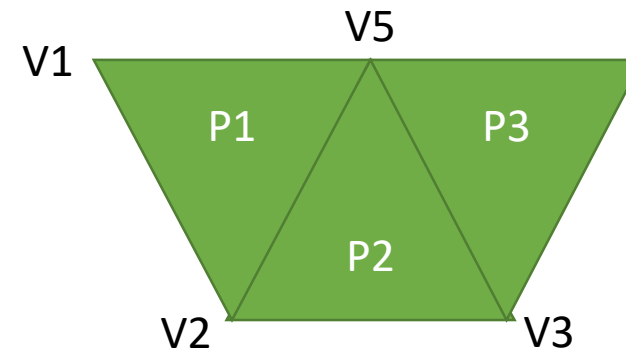
$$P = \{ (X_1, Y_1, Z_1), (X_2, Y_2, Z_2), \dots, (X_n, Y_n, Z_n) \}$$

- Each polygon represent by a list of vertex of coordinate

$$P1 = \{ V1, V2, V5 \}$$

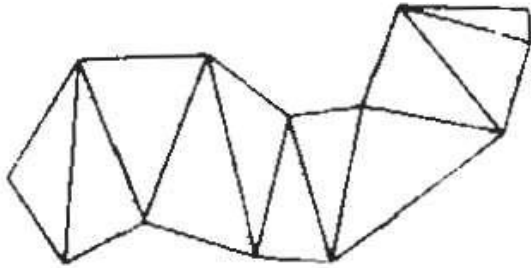
$$P2 = \{ V2, V3, V5 \}$$

$$P3 = \{ V3, V4, V5 \}$$

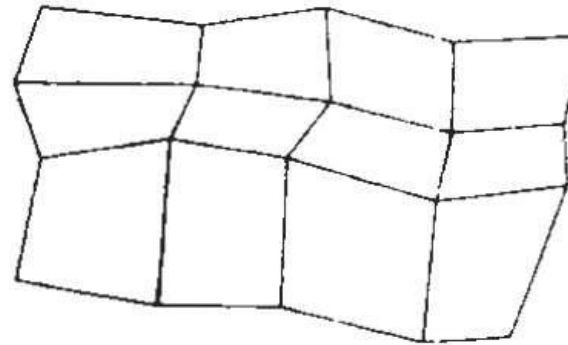


3D Object Representation

b) Polygon Meshes



A triangle strip formed with 11 triangle mesh connecting 13 vertices

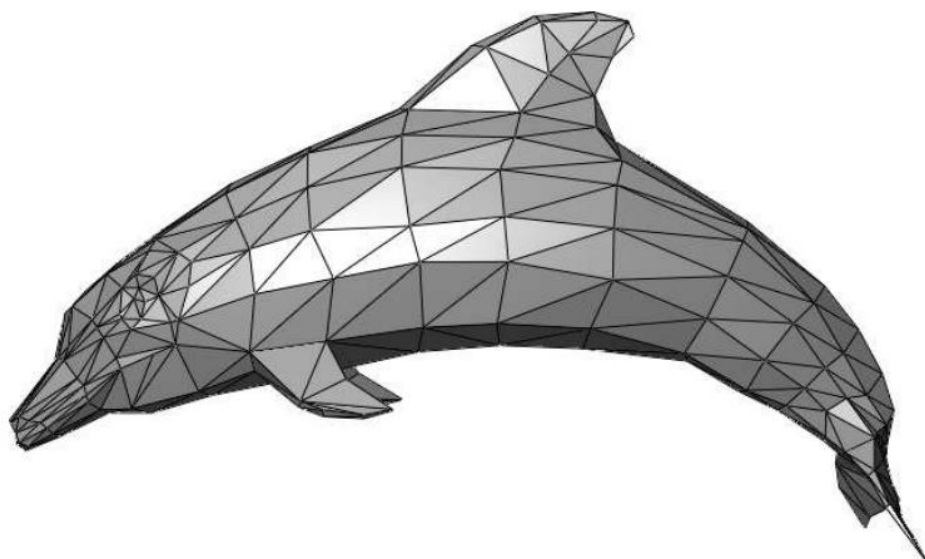
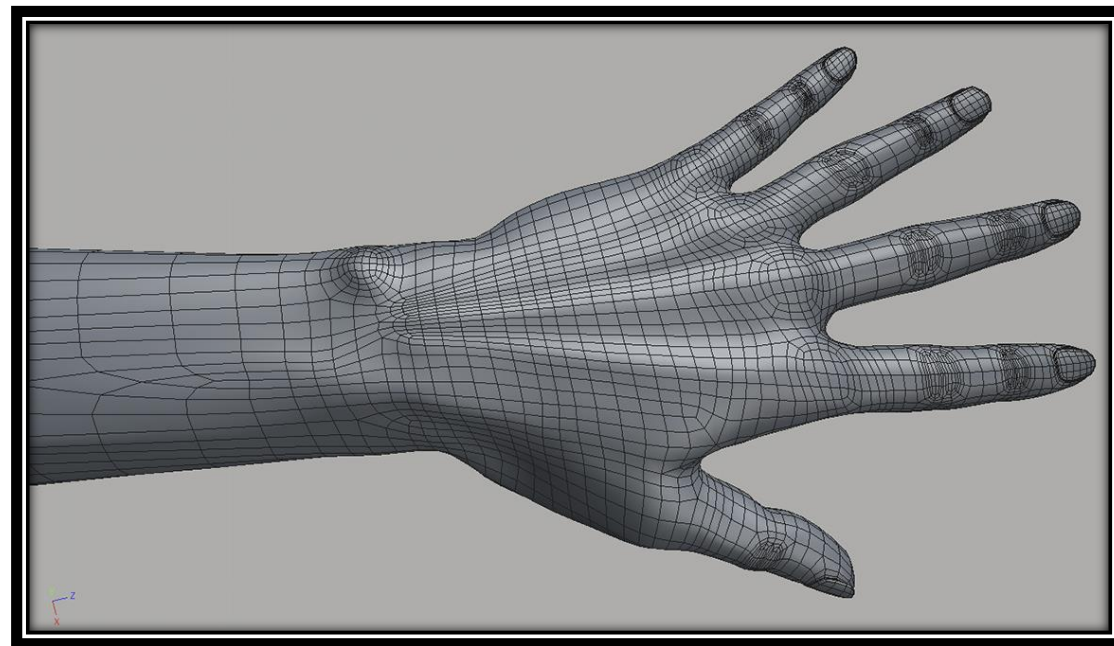
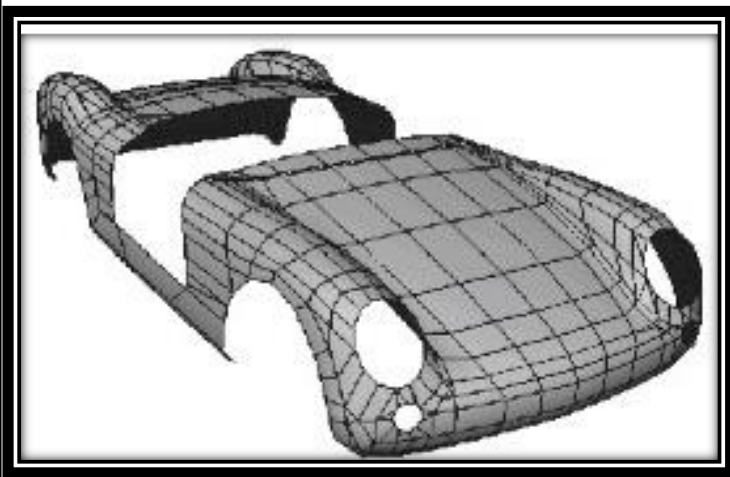
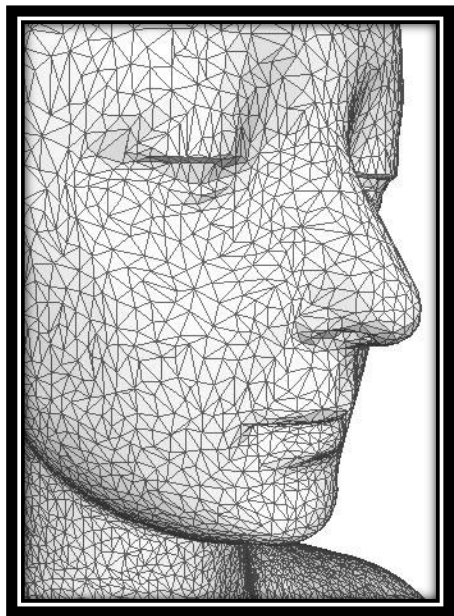


A quadrilateral mesh containing 12 quadrilaterals constructed from a 5 by 4 input vertex array

- When polygons are specified with more than 3 vertices, it is possible that the vertices may not all lie in one plane.
- This can be due to numerical error or errors in selecting coordinate positions for the vertices

b) Polygon Meshes

- Remedies:
 - i. Simply divide the polygons into triangles
 - ii. Approximate the plane parameters ABC and project in same plane (i.e. approximate A to yz plane, B to xz plane, C to xy plane etc)
- High quality graphics systems typically model objects with polygon meshes and set up a database of geometric and attribute information to facilitate processing of the polygon facets



Polygon Meshes

c) Plane Equation

- It is used to determine the spatial orientation of the individual surface component of the object. The equation for a plane surface can be expressed in the form

$$Ax + By + Cz + D = 0 \text{ where } (x, y, z) \text{ is any point on the plane,}$$

and the coefficients A , B , C , and D are constants describing the spatial properties of the plane

Let (x_1, y_1, z_1) , (x_2, y_2, z_2) , and (x_3, y_3, z_3) be three successive polygon vertices of the polygon.

$$Ax_1 + By_1 + Cz_1 + D = 0$$

$$Ax_2 + By_2 + Cz_2 + D = 0$$

$$Ax_3 + By_3 + Cz_3 + D = 0$$

Using Cramer's Rule

$$Ax_1 + By_1 + Cz_1 + D = 0,$$

$$Ax_2 + By_2 + Cz_2 + D = 0,$$

$$Ax_3 + By_3 + Cz_3 + D = 0$$

$$A = \begin{vmatrix} 1 & y_1 & z_1 \\ 1 & y_2 & z_2 \\ 1 & y_3 & z_3 \end{vmatrix} \quad B = \begin{vmatrix} x_1 & 1 & z_1 \\ x_2 & 1 & z_2 \\ x_3 & 1 & z_3 \end{vmatrix} \quad C = \begin{vmatrix} x_1 & y_1 & 1 \\ x_2 & y_2 & 1 \\ x_3 & y_3 & 1 \end{vmatrix}$$

-

Expanding the determinant we can write that,

$$A = y_1(z_2 - z_3) + y_2(z_3 - z_1) + y_3(z_1 - z_2)$$

$$B = z_1(x_2 - x_3) + z_2(x_3 - x_1) + z_3(x_1 - x_2)$$

$$C = x_1(y_2 - y_3) + x_2(y_3 - y_1) + x_3(y_1 - y_2)$$

$$D = -x_1(y_2 z_3 - y_3 z_2) - x_2(y_3 z_1 - y_1 z_3) - x_3(y_1 z_2 - y_2 z_1)$$

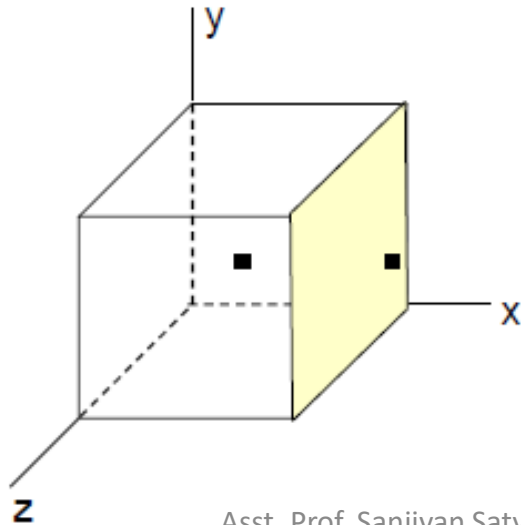
- ❑ Plane equations can be used to identify the position of spatial points relative to the polygon facets of an object
- ❑ For any point (x, y, z) not on a plane with parameters A, B, C, D , we have

$$A x + B y + C z + D \neq 0,$$

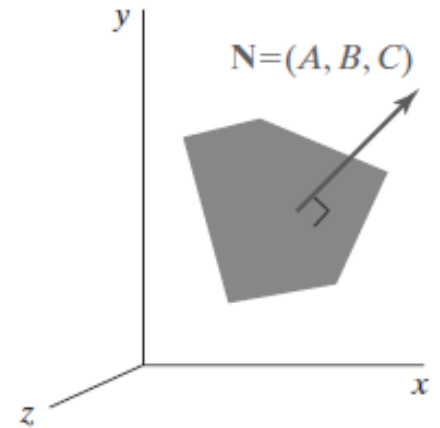
- **Inside outside tests of the surface:**

$A x + B y + C z + D < 0$, point (X, Y, Z) is inside the surface (behind the plane)

$A x + B y + C z + D > 0$, point (X, Y, Z) is outside the surface (in front of the plane)



- The orientation of a polygon surface in space is described using a **normal vector**. The normal vector:
 - ❑ Points from the back face of the polygon to the front face.
 - ❑ Is perpendicular to the plane of the polygon.
- To obtain the normal vector, a vector cross product is used:



- ❑ Select any three vertices V_1, V_2, V_3 of the polygon in counter clockwise order
- ❑ Form two vectors: one from V_1 to V_2 and another from V_1 to V_3
- ❑ Calculate the normal vector N as the cross product of these two vectors.

$$N = (V_2 - V_1) \times (V_3 - V_1)$$

$$\mathbf{N} = (\mathbf{V}_2 - \mathbf{V}_1) \times (\mathbf{V}_3 - \mathbf{V}_1)$$

plane can be represented as:

$$(A, B, C) = \mathbf{N}$$

$$\mathbf{N} \cdot (x, y, z) + D = 0$$

$\mathbf{P}$ is a point in the surface (i.e. a vertex)

$$D = -\mathbf{N} \cdot \mathbf{P}$$

2. Quadratic Surface

- A frequently used class of objects are the quadric surfaces, which are described with **second-degree equations (quadratics)**
- They include
Spheres, Ellipsoid, Paraboloids, Hyperboloids etc.

a. Sphere:

For a set of surface point (x, y, z) spherical surface is represented by equation,

$$(x - x_c)^2 + (y - y_c)^2 + (z - z_c)^2 = r^2$$

Where, (x_c, y_c, z_c) is the center of sphere and r is the radius of the sphere.

b. Ellipsoid:

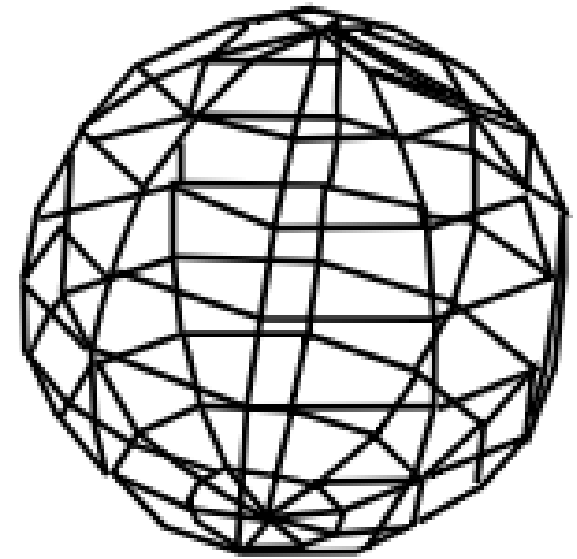
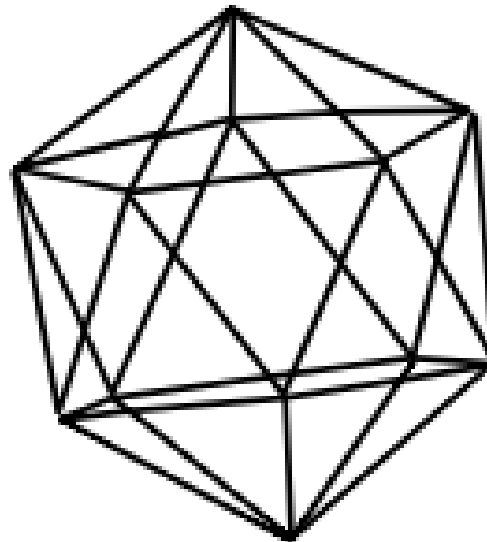
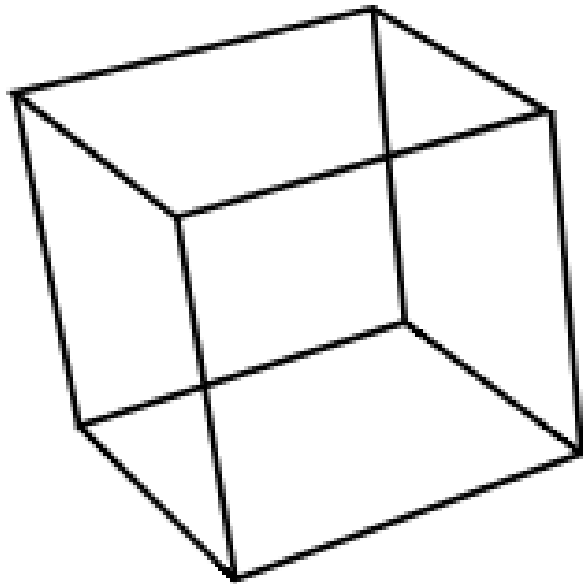
If (x, y, z) be the any point on ellipsoid with the radius a, b & c along x, y & z axis then represented as;

$$\frac{x^2}{a^2} + \frac{y^2}{b^2} + \frac{z^2}{c^2} = 1$$

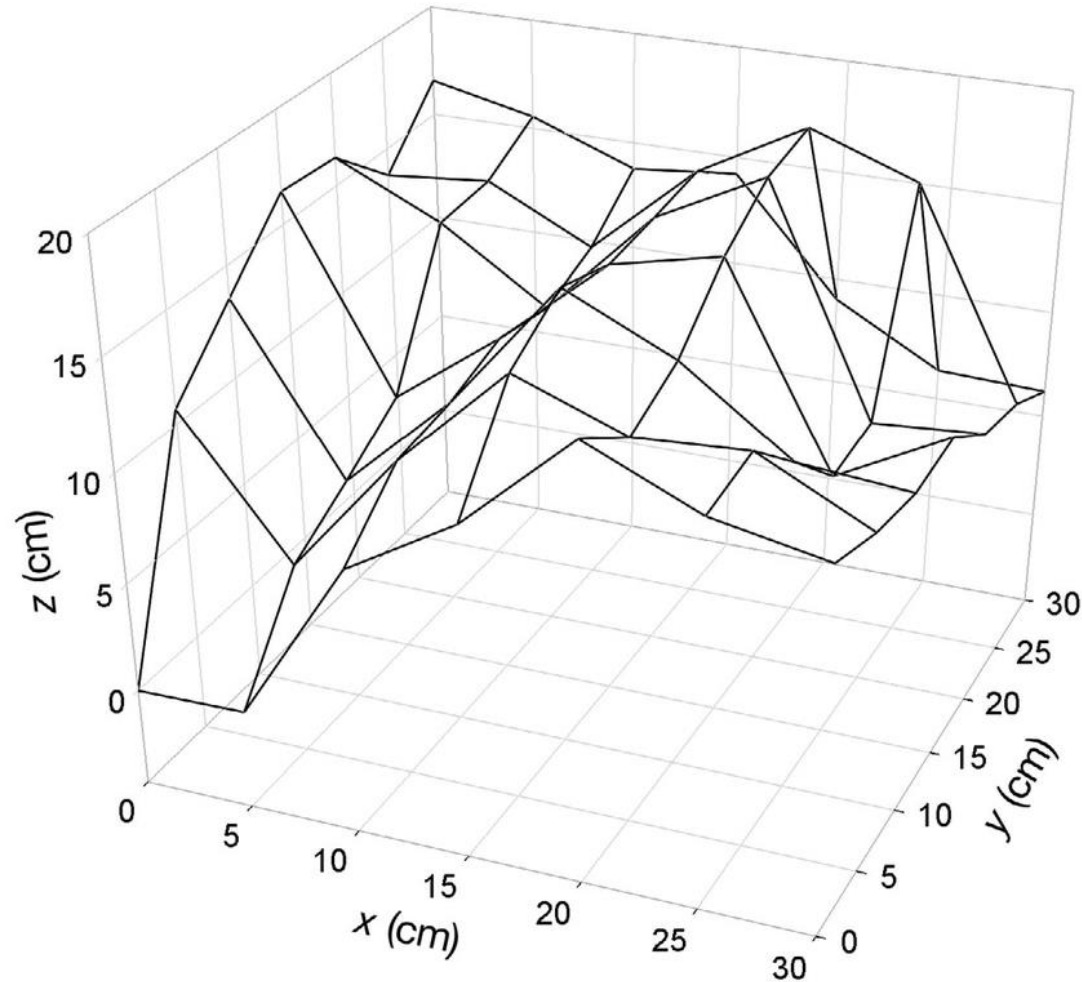
3. Wireframe

- ❑ If an object may be represented through the collection of points and line then it's called wireframe object
- ❑ It's used to define skeleton of 3D objects in terms of points and lines
- ❑ Mostly use in field of engineering drawings to represent the drawing of structure and missing parts
- ❑ Wireframe represented object requires less memory for storage and fast for display and surface rendering
- ❑ Wireframes provide the most basic representation of a three-dimensional scene or object.
- ❑ They are often used as the starting point in 3D modeling since they create a "frame" for 3D structures

- ❑ For example, a 3D graphic designer can create a model from scratch by simply defining points (vertices) and connecting them with lines (paths).
- ❑ Once the shape is created, surfaces or textures can be added to make the model appear more realistic.
- ❑ The lines within a wireframe connect to create polygons, such as triangles and rectangles, that together represent three-dimensional shapes
- ❑ The result may be as simple as a cube or as complex as a three-dimensional scene with people and objects.

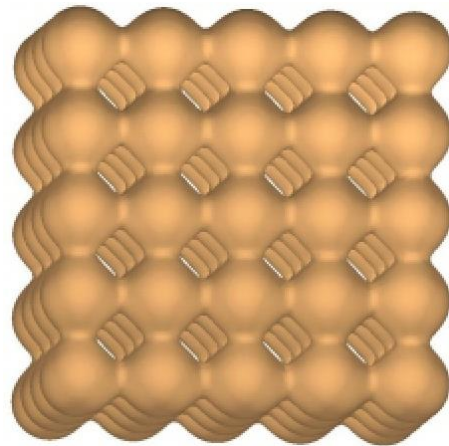


The number of polygons within a wireframe is typically a good indicator of how detailed the 3D model is.

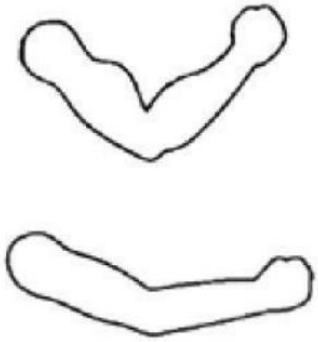


4. Blobby objects

- ❑ Object that don't maintain a fixed shape but changes their surface characteristics during motion or closer to another object are called blobby object
- ❑ These objects are referred to as blobby objects, **since their shapes show a certain degree of fluidity**
- ❑ E.g. molecular structure, water droplet, muscle shape in human body etc.

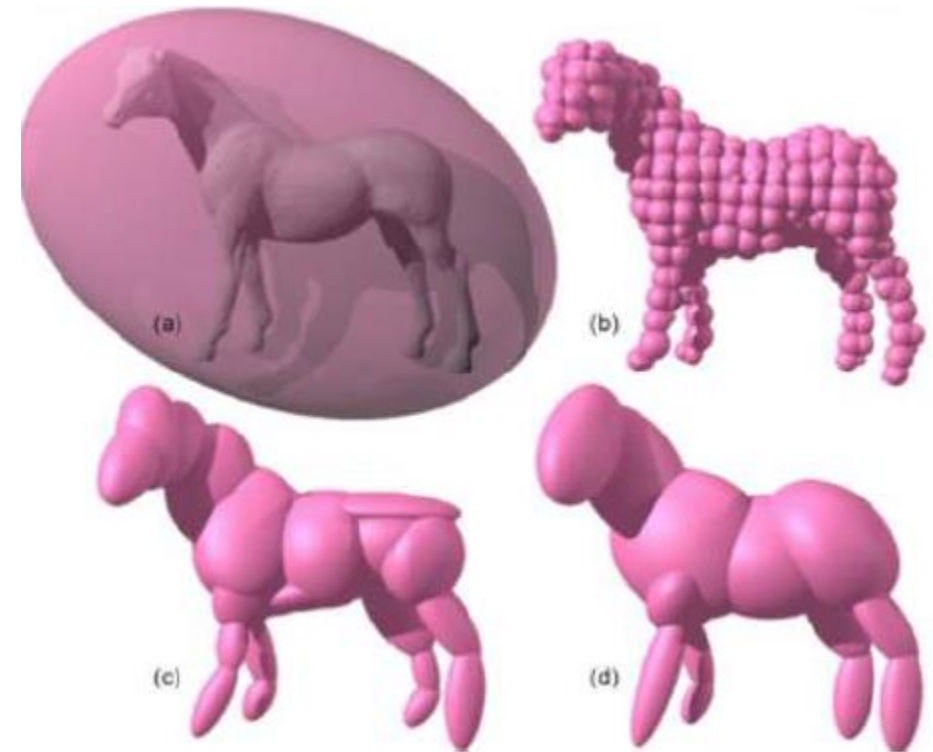


- ❑ Several models have been developed for representing blobby objects as distribution functions over a region of space
- ❑ One technique is to use a combination of Gaussian density functions (Gaussian bumps)



Blobby muscles in human arm

Blobby objects



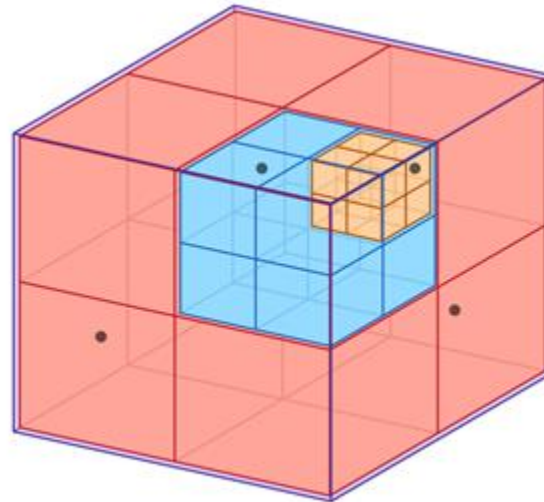
5. Spline representation

- Spline curve: Curve consisting of continuous curve segments approximated or interpolated on polygon control points
- Spline surface: a set of two spline curves matched on a smooth surface.

(Studied Earlier)

b) Space partitioning representation

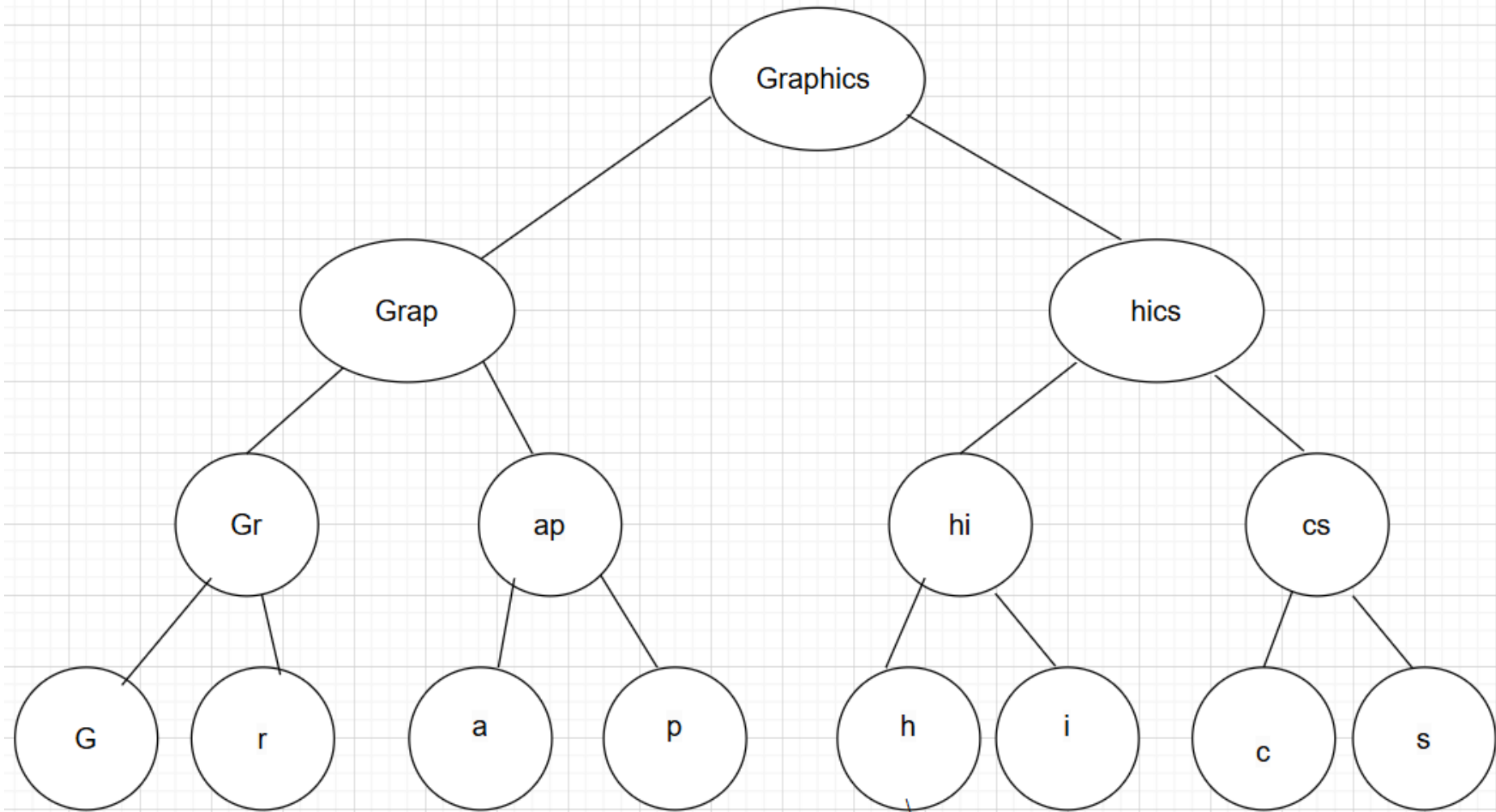
- A "space partitioning representation" is a method in computer graphics and geometry where a 3D object is described by dividing the space it occupies into smaller, non-overlapping regions, effectively defining its interior properties by partitioning the surrounding space rather than just its surface boundaries; essentially, it's a way to represent an object by describing the space it takes up within a larger volume, often using data structures like octrees or quadtrees to store this information.



1. Binary Space Partitioning

- ❑ Binary space partitioning arose in the context of 3D Computer Graphics in 1969, where the structure of a BSP tree allows for spatial information about the objects in a scene that is useful in rendering, such as objects being ordered from front-to-back with respect to a viewer at a given location, to be accessed rapidly.
- ❑ Binary space partitioning is a generic process of recursively dividing a scene into two until the partitioning satisfies one or more requirements. It can be seen as a generalization of other spatial tree structures such as k-d trees and quadrees, one where hyper plane that partition the space may have any orientation, rather than being aligned with the coordinate axes as they are in k-d trees or quadrees.
- ❑ When used in computer graphics to render scenes composed of planar polygons, the partitioning planes are frequently chosen to coincide with the planes defined by polygons in the scene.

- ❑ A BSP tree is a way of grouping data so it can be processed faster. It's used in many computer science applications, not just VFX rendering.
- ❑ BSP trees are often used by 3D video games, particularly first-person shooters and those with indoor environments
- ❑ The grouping process starts with data of some kind, and divides it into two parts. Then the two parts are divided into two parts, and so on until the parts at the end contain the smallest piece of working data.
- ❑ As a very simple example, let's take the word "graphics" and partition it with a BSP tree. For this example, a single letter of the alphabet is the smallest possible element. BSP trees are very useful for real time interaction with displays of static images. Before the static scene is rendered, the BSP tree is calculated. BSP trees can be traversed very quickly (linear time) for hidden surface removal and shadow casting. With some work, BSP trees can be modified to handle dynamic events in a scene



Simple BSP Tree

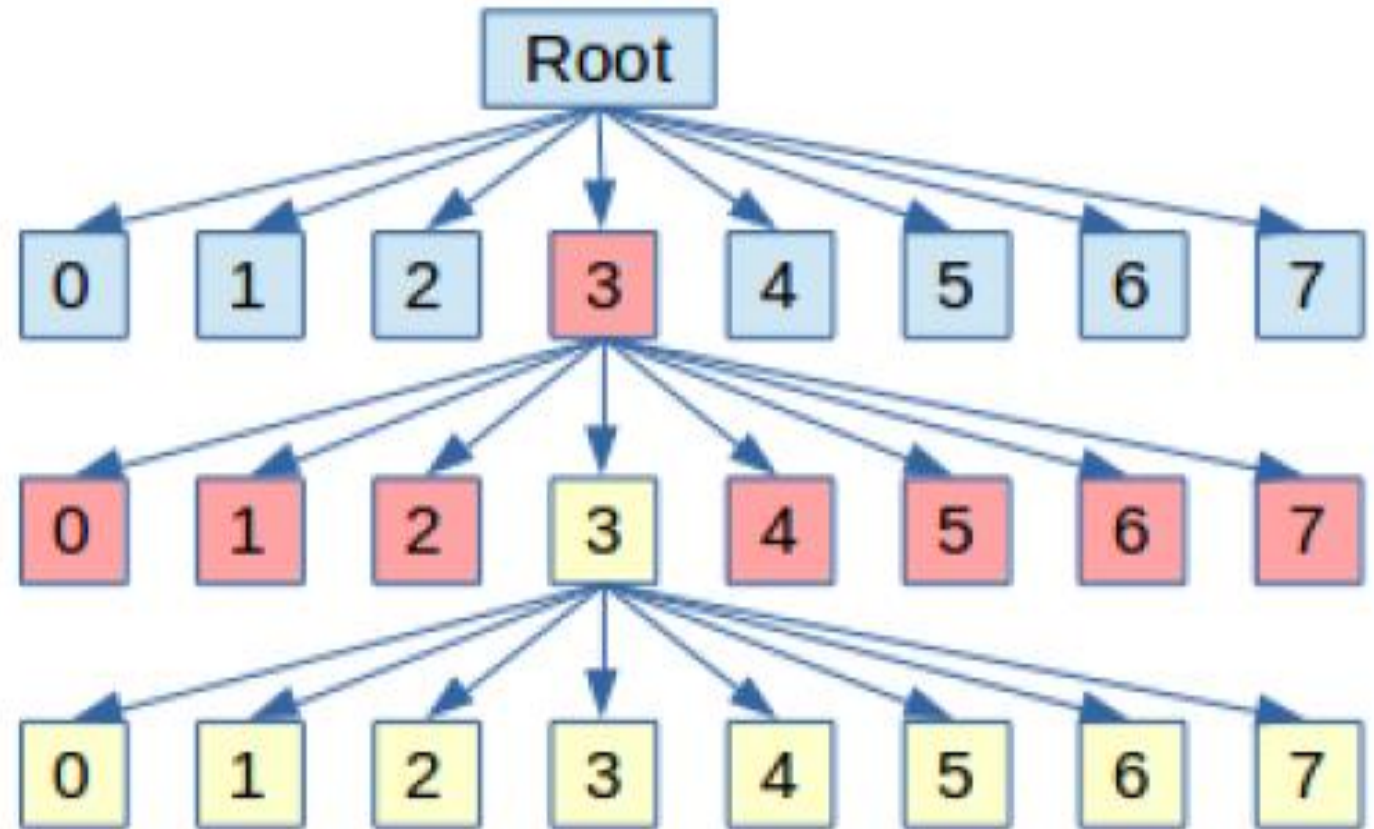
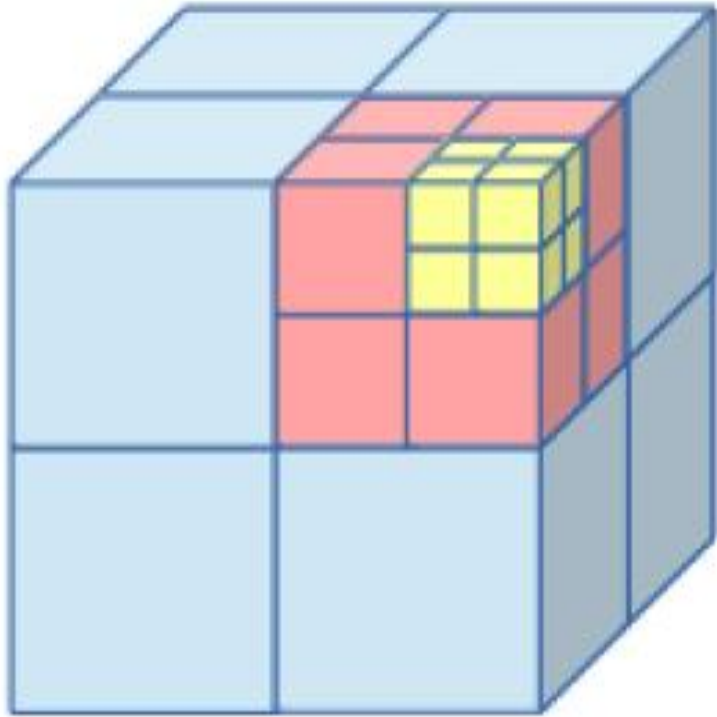
Asst. Prof. Sanjivan Satyal

2. Octree

- ❑ Octree is a tree data structure in which each internal node can have at most 8 children.
- ❑ Like Binary tree which divides the space into two segments, Octree divides the space into at most eight-part which is called as octanes.
- ❑ It is used to store the 3-D point which takes a large amount of space.
- ❑ If all the internal node of the Octree contains exactly 8 children, then it is called full Octree.
- ❑ It is also useful for high-resolution graphics like 3D computer graphics.

The Octree can be formed from 3D volume by doing the following steps:

- ❑ Divide the current 3D volume into eight boxes
- ❑ If any box has more than one point then divide it further into boxes
- ❑ Do not divide the box which has one or zero points in it
- ❑ Do this process repeatedly until all the box contains one or zero point in it



Thank you